

OpenWAM: An Open, Modular Exploration Towards Systematic World-Action Model Pretraining

Yuran Wang^{1,*}, Siqiao Huang^{2,*}, Mingleyang Li^{3,*}, Chenhao Zhang^{3,*}, Jiaqi Liang^{3,*}, Weiyang Jin⁴, Yue Chen³, Xuemin Chi⁵, Donghao Zhou⁶, Qize Yu³, Yu-Kai Wang³, Yuhao Rui³, Shenzhe Yao², Zhen Yuan⁴, Zhenhao Shen³, Kefei Zhu³, Zijie Zhu⁴, Ning Gao⁷, Xiaowei Chi³, Guanqi He², Shanghang Zhang³, Hao Dong³, Lin Shao^{1,†}, Hang Zhao^{2,†}

¹National University of Singapore, ²Tsinghua University, ³Peking University, ⁴The University of Hong Kong, ⁵Zhejiang University, ⁶The Chinese University of Hong Kong, ⁷Shanghai Jiao Tong University
*Equal contribution, †Project lead, ‡Equal advising

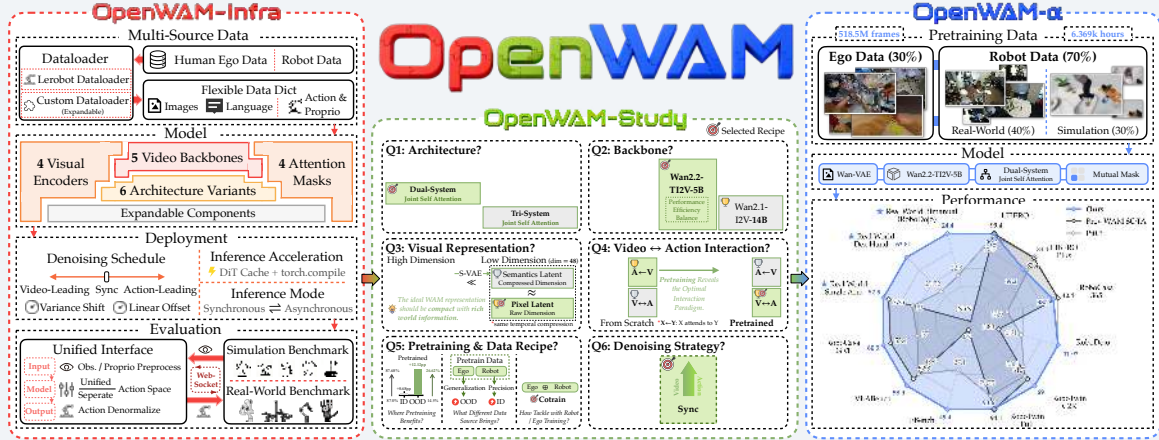


Figure 1 Overview of OpenWAM. **OpenWAM-Infra** (left) factorizes world-action modeling into composable modules with unified training, deployment, and evaluation; **OpenWAM-Study** (middle) resolves the design space through controlled questions and distills a pretraining recipe; **OpenWAM-α** (right), pretrained on 518.5M frames of egocentric and robot data, sustains top-tier performance from simulation to the real world.

Project Page: <https://openwam-official.github.io/>

Code: <https://github.com/OpenWAM-Official/OpenWAM>

Model & Data: <https://huggingface.co/OpenWAM>

Abstract

World-Action Models inherit world knowledge from video-generative priors, and channel it into executable control signals through embodied experience. Existing systems, however, are monolithic: the generative backbone, visual representation, architecture, information flow, inference procedure, and training data are tightly coupled, obscuring which design choices matter and why. We introduce **OpenWAM**, an open research stack that turns world-action pretraining into a controlled experimental program. **OpenWAM-Infra** factorizes the WAM design space into composable modules with unified training, inference, deployment, and evaluation. On this substrate, **OpenWAM-Study** examines three questions through controlled experiments: what to inherit, how world and action learning interact, and how their synergy scales; and distills three principles: upstream knowledge transfers through a sufficiently capable generative backbone and a compact, information-rich latent space; world-action synergy requires dedicated action capacity, explicit world-to-action information flow, and synchronized joint denoising; and embodied pretraining principally improves out-of-domain generalization, with one-stage co-training over egocentric and robot data integrating world coverage and action grounding. Composing these principles, we build **OpenWAM-α**, an open WAM pretrained on roughly 6,400 hours of egocentric human and robot data and evaluated across simulation and real-world benchmarks. Across the eight simulation benchmarks and the real-robot experiments, which together span embodiments from single-arm and bimanual manipulation to dexterous hands, **OpenWAM-α** delivers consistently excellent performance, sustaining its top-tier standing from simulation to the physical world. We release the full stack, including infrastructure, evaluation protocols, pretrained models, and data recipes, to facilitate future research.

Contents

1	Introduction	3
2	Related Work	4
3	OpenWAM-Infra: A Modular Infrastructure for World–Action Modeling	5
3.1	Composable Model	5
3.2	Training Runtime	7
3.3	Deployment Runtime	8
3.4	Evaluation Protocol	9
4	OpenWAM-Study: Design Principles for World–Action Models	11
4.1	Inheriting Upstream World Knowledge	11
4.1.1	Generative World Priors	11
4.1.2	Visual Representation Priors	12
4.2	Building Synergy between World and Action Learning	13
4.2.1	Architectural Capacity	13
4.2.2	Training-Time Information Flow	13
4.2.3	Inference-Time Information Flow	14
4.3	Consolidating Knowledge Across Domains	15
4.3.1	Problem Setup and Evaluation Protocol	15
4.3.2	Embodied Pretraining Primarily Expands OOD Generalization	15
4.3.3	Pretraining Changes the Preferred Information Flow	16
4.4	Concluding Remarks	16
5	OpenWAM-α: From Principles to a Pretrained Model	17
5.1	OpenWAM- α Architecture, Training, and Deployment	17
5.1.1	Architecture	18
5.1.2	Training	18
5.1.3	Deployment	18
5.2	Multi-Domain Pretraining Data and Curation	19
5.2.1	Pretraining Data Mixture	19
5.2.2	Data Curation	20
5.3	Simulation Benchmark Evaluation	20
5.3.1	How Does OpenWAM- α Perform?	21
5.3.2	VLA versus WAM: Which Paradigm Prevails?	23
5.4	Real-Robot Evaluation	24
6	Conclusions	26
	References	27
	Appendix	33
A	Limitations and Future Work	33
B	Training Details	33
C	Real-World Evaluation Protocols	35
D	Per-Benchmark Simulation Results	39

1 Introduction

“Knowledge is the beginning of action, action is the completion of knowledge.”

— Yang-ming Wang, *“Instructions for Practical Living”* (Wang, 1963)

Intelligence requires more than recognizing the world: an embodied agent must anticipate how the world changes and act to bring about desired changes. Modern vision and vision–language models have learned rich semantic representations from large-scale image–text data (Radford et al., 2021; Caron et al., 2021; Tong et al., 2024). Video generation models go one step further by learning to synthesize how visual worlds may evolve over time (Ho et al., 2022; Brooks et al., 2024). An embodied system, however, must learn not only *what can happen in the world*, but also *what actions it can take to make it happen*.

This distinction exposes a fundamental data asymmetry in embodied learning. Videos of the changing world are abundant, whereas robot trajectories with executable action labels remain comparatively scarce (Ye et al., 2026d). Earlier approaches such as ACT (Zhao et al., 2023) and Diffusion Policy (Chi et al., 2025) largely learn visual regularities and control together solely from robot demonstrations. More recently, Vision–Language–Action (VLA) models (Brohan et al., 2023; Kim et al., 2024; Black et al., 2024) instead inherit semantic and linguistic knowledge from pretrained vision–language models. World–Action Models (WAMs) (Pai et al., 2025; Ye et al., 2026c; Li et al., 2026b) offer a different warm start: they inherit a generative prior over visual dynamics from video generation pretraining and adapt it through embodied experience. Because video generation is explicitly trained to model temporal evolution, it provides a direct starting point for learning how actions interact with physical change. In this sense, a WAM *inherits world knowledge from video generation priors, then learns how to take actions in this world through embodied experience*.

The promise of a WAM, however, lies not merely in initializing a policy with a video model or attaching an action head to a video generator. Its central hypothesis is that world prediction and action generation can be learned in *synergy*: world modeling supplies structured knowledge of states, dynamics, and possible futures that can inform action, while action learning focuses the model on changes that matter for control. However, realizing this synergy is nontrivial. Useful knowledge may reside in different parts of an upstream video model; nominally joint world and action prediction may still lack an effective information path; and a design that fits one training domain may fail to retain its advantage across new scenes or embodiments. These challenges lead to three central questions:

Question 1: What world knowledge should a WAM inherit?

Question 2: How can we create synergy between world and action learning?

Question 3: How can this synergy be scaled across domains?

Answering these questions is difficult with existing monolithic systems, where the generative backbone, visual representation, model architecture, information flow, inference procedure, and composition of training data are often tightly coupled (Kim et al., 2026b; Bi et al., 2026; Zhang et al., 2026b). We therefore introduce **OpenWAM**, an open research stack for systematically developing World–Action Models. **OpenWAM-Infra** factorizes the WAM design space into modular components, while providing unified training, inference, deployment, and evaluation across domains and embodiments (Section 3). This modularity turns world–action modeling from a collection of coupled implementation choices into a controlled experimental program.

Building on this framework, **OpenWAM-Study** investigates the design principles underlying World–Action Models (Section 4). Our study produces three main findings. First, upstream world knowledge transfers most effectively through a sufficiently capable generative backbone and a compact, information-rich visual latent space. Second, world–action synergy does not emerge from parameter count or joint prediction alone. It is best fostered with sufficient action-specific capacity, explicit world-to-action information flow, and a joint test-time denoising schedule. Third, embodied pretraining primarily improves out-of-domain generalization rather than in-domain fitting: human egocentric video broadens world coverage, robot trajectories provide executable action knowledge, and their joint training offers the strongest practical integration strategy in our experiments. Together, these findings suggest a practical recipe from *inheriting world knowledge*, to *enabling world–action synergy*, to *scaling that synergy across domains*.

Finally, we compose the resulting design principles into **OpenWAM- α** , an open pretrained World–Action Model (Section 5). Pretrained on 518M frames ($\approx 6,400$ hours) of egocentric human and robot data through a unified 80-D action space, OpenWAM- α demonstrates that the recipe distilled from controlled settings remains effective when scaled across heterogeneous data, domains, and embodiments. It delivers consistently strong results on eight simulation benchmarks covering five embodiment categories, and preserves this standing in real-robot experiments on single-arm, bimanual, and dexterous-hand platforms. Beyond the scores themselves, these large-scale evaluations also distill further insights into the design and scaling behavior of embodied foundation models. To let the community reproduce and extend these findings, we release alongside the model the full stack: the infrastructure, evaluation protocols, pretrained weights, and data recipes.

In summary, our major contributions are as follows:

- **OpenWAM-Infra: a Modular Infrastructure for World–Action Modeling.** It factorizes model, representation, training, inference, deployment, and evaluation choices, enabling controlled comparison across WAM designs and embodiments (Section 3).
- **OpenWAM-Study: Design Principles for World–Action Synergy.** Through controlled studies of upstream priors, architectural capacity, information flow, denoising, and multi-domain pretraining, we identify how world knowledge can interact productively with action learning (Section 4).
- **OpenWAM- α : a Pretrained World–Action Model.** It instantiates and scales the derived principles into an open model for evaluating generalization and efficiency across domains and embodiments (Section 5).

2 Related Work

World–Action Models. World models (LeCun et al., 2022; Ha & Schmidhuber, 2018) learn predictive structure from observations and have long supported control through planning (Zhou et al., 2024; Maes et al., 2026; Huang et al., 2026), model-based reinforcement learning (Hafner et al., 2019; M. Moerland et al., 2023), and policy evaluation (Huang et al., 2025; Wang et al., 2026b). World–Action Models (WAMs) (Ye et al., 2026c; Pai et al., 2025; Li et al., 2026b) more directly connect this predictive capacity to executable behavior by serving as a policy model. Whereas VLA models (Brohan et al., 2023; Kim et al., 2024; Black et al., 2024) primarily inherit semantic and linguistic knowledge from vision–language pretraining (Beyer et al., 2024; Bai et al., 2025), WAMs initialize from video-generative priors so that action learning begins with a model with rich visual dynamical priors (Wan et al., 2025; Ali et al., 2025). Yet existing systems remain largely *monolithic*, where changes in model architecture, training procedure, data recipe, and sampling schedule are often coupled. Consequently, it remains unclear which components transfer world knowledge, which interactions create world–action synergy, and which benefits persist across domains. **OpenWAM** exposes these coupled choices as controlled variables and organizes them around precisely these three questions.

Open Research Ecosystems for Generalist Robot Policy Learning. Open models and codebases have made generalist robot learning increasingly accessible. OpenVLA (Kim et al., 2024) established an open-weight pretrained baseline, while StarVLA (Community, 2026) provides a modular and performant platform for varied design choices. StarVLA- α (Ye et al., 2026b) complements this breadth with a pretrained model of minimalist design, and XPolicyLab (Community et al., 2026) contributes a unified standard and open ecosystem for policy evaluation and deployment. These efforts have significantly reduced development complexity in the VLA research community; however, in the WAM community, such open research ecosystems remain largely absent. A modular system in this realm accompanied by a strong pretrained model would help democratize research, as well as serve as a principled foundation for understanding and scaling world–action model pretraining.

Towards a Scientific Understanding of Model Design. A growing line of work treats model design as an empirical science (Allen-Zhu, 2026; Karras et al., 2022; McKinzie et al., 2024; Liu et al., 2022; Wen et al., 2026): decomposing a complex system into controlled variables, testing the mechanisms behind observed gains, deriving a recipe, and validating whether it survives scale. In multimodal learning, Cambrian-1 (Tong et al., 2024) and Beyond Language Modeling (Tong et al., 2026) systematically study visual representations, modality-specific capacity, data composition, and unified pretraining; Towards Physics of Multimodal Pretraining (Han et al., 2026) further isolates knowledge flow, synergy versus competition, and the timing of modality unification. In robot learning, analyses around Action Chunking (Simchowit et al., 2025; Zhang et al., 2025b; Lazzati

et al., 2026) and Generative Control Policies (Pan et al., 2026) have substantially reshaped the community’s understanding of these topics. At the data and system level, Large Behavior Models (Barreiros et al., 2026), LBM co-train (Lin et al., 2026a), StarVLA- α (Ye et al., 2026b), and OpenHLM (Hu et al., 2026) similarly use controlled comparisons to study multitask transfer, heterogeneous supervision, action design, and embodiment interfaces. **OpenWAM** brings this methodology to WAMs: **OpenWAM-Infra** builds the substrate for controlled experiments, **OpenWAM-Study** turns them into controlled scientific questions about inheritance, synergy, and scaling, and **OpenWAM- α** scales the resulting recipe under heterogeneous multi-domain pretraining.

3 OpenWAM-Infra: A Modular Infrastructure for World-Action Modeling

Most existing world-action models differ substantially in architecture and infrastructure implementation, with no shared standard; since each system is built around a single model design, its model, training, serving, and evaluation components are likewise organized idiosyncratically and are often tightly coupled. This brings two problems: 1) such codebases are difficult for users to extend or build upon, and 2) the coupling among components allows modules to interfere with one another, confounding the conclusions drawn from controlled comparisons. **OpenWAM-Infra** addresses both problems by factoring world-action modeling into four decoupled components with explicit interfaces: a *composable model* assembled from interchangeable encoders, stream backbones, and visibility attention masks (Section 3.1); a *training runtime* that trains every such model with one trainer (Section 3.2); a *deployment runtime* that serves every resulting checkpoint through one policy server (Section 3.3); and an *evaluation protocol* through which every benchmark reaches that server (Section 3.4). These components are either mutually independent or related by strict one-way dependencies, which keeps the codebase straightforward to extend, insulates modules from mutual interference, and further provides the substrate on which **OpenWAM-Study** (Section 4) conducts controlled experiments and **OpenWAM- α** (Section 5) is instantiated at scale.

3.1 Composable Model

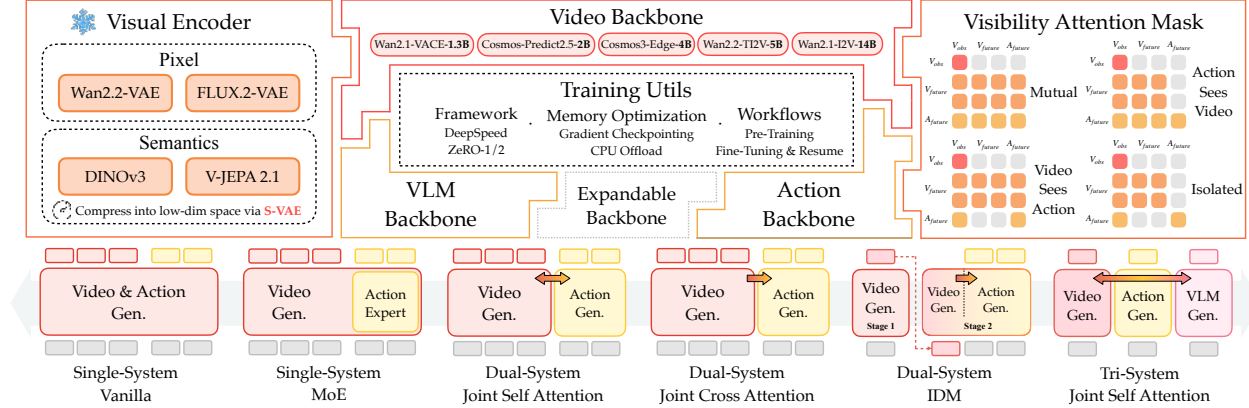


Figure 2 OpenWAM Model Infra. *Top:* the three classes of interchangeable modules: the visual encoder $\mathcal{E}$ (left); the stream backbones $\mathcal{S}$ (middle); and the visibility attention mask $\mathcal{M}$ (right). The central Training Utils panel summarizes the utilities of the training runtime (Section 3.2). *Bottom:* the composition rule C assembles the modules into six architecture variants across the Single-System, Dual-System, and Tri-System families.

As shown in Figure 2, OpenWAM-Infra organizes a World-Action Model (WAM) as three classes of interchangeable modules that a composition rule C assembles into a concrete **architecture**, written $C(\mathcal{E}, \mathcal{S}, \mathcal{M})$:

- **Visual Encoder $\mathcal{E}$:** It maps observations into the latent sequences the world stream predicts;
- **Stream Backbones $\mathcal{S}$:** It processes the model’s token streams: a world stream $\mathcal{W}$, an action stream $\mathcal{A}$, optionally an understanding stream $\mathcal{U}$, and any further streams a design may introduce; distinct streams may share a single backbone;

- **Visibility Attention Mask $\mathcal{M}$:** It specifies the attention relations among streams and tokens, i.e. which tokens may attend to which, both within and across streams.

Visual Encoder. The encoder $\mathcal{E}$ maps observations into the latent sequence the world stream predicts and is always kept frozen; different encoders capture different information in their latents and in turn induce different world-stream behavior. In most cases, a video backbone is accompanied by a natively matched encoder, in which case the pretrained parameters of the base DiT are reused directly, retaining the full benefit of pretraining. Beyond this default, OpenWAM-Infra additionally supports pluggable encoders: swapping $\mathcal{E}$ alters the latent representation to be predicted, and the base DiT can further be re-initialized to exclude the influence of pretrained parameters. Currently, OpenWAM-Infra provides two reconstructive encoders, trained on pixel-reconstruction objectives: Wan2.2-VAE (Wan et al., 2025) and FLUX.2-VAE (Black Forest Labs, 2025), and two representation encoders (Zheng et al., 2026a): DINOv3 (Siméoni et al., 2025) and V-JEPA 2.1 (Mur-Labadia et al., 2026), along with an optional S-VAE (Zhang et al., 2025a) module that compresses the latent dimension. These capabilities together support the study of visual representations in Section 4.1.2.

Stream Backbones. The backbones in $\mathcal{S}$ are laid out in and around the central panel of Figure 2. The *video backbone* ($\mathcal{W}$) predicts the temporal evolution of future world visual states, while the *action backbone* ($\mathcal{A}$) predicts the upcoming action chunk (Zhao et al., 2023); together they form the world-action core of the model. The optional *VLM backbone* ($\mathcal{U}$) supplements this core with semantic understanding of the current observation. Beyond these, the backbone roster is itself **expandable**: further backbones can be registered to execute any additional streams a design may introduce. Regardless of type, every backbone executes through one code contract that decomposes its forward pass into three stages:

- **prepare**, invoked once before the stack, which embeds the inputs into the initial token state;
- **per-layer block step**, invoked once per layer, which advances this state through one transformer layer;
- **finalize**, invoked once after the stack, which maps the final state to the stream’s prediction.

Each layer may further split its block step into a **pre-attention** half, which emits the layer’s queries, keys, and values, and a **post-attention** half, which consumes the attention output, so that the attention between the two halves can be computed jointly across streams.

For the video backbone, OpenWAM-Infra supports five pretrained video generation models with increasing model parameters, namely Wan2.1-VACE-1.3B, Cosmos-Predict2.5-2B, Cosmos3-Edge-4B, Wan2.2-TI2V-5B, and Wan2.1-I2V-14B (Wan et al., 2025; Ali et al., 2025). For VLM backbones, OpenWAM-Infra currently supports only the Qwen3-VL family (Bai et al., 2025). For the action backbone, OpenWAM-Infra offers two options: a separate set of parameters residing in ActionDiT, or a shared video backbone in which action tokens join the video token sequence and are processed jointly.

Visibility Attention Mask. The mask $\mathcal{M}$ governs the information flow with the mixed self-attention through which streams interact: it factorizes into intra-modality and cross-modality blocks, granting attention where tokens reinforce one another and withholding it where their mutual influence must be isolated. Over the video and action modalities, this factorization reads

$$\mathcal{M} = \begin{pmatrix} \mathcal{M}_{V \leftarrow V} & \mathcal{M}_{V \leftarrow A} \\ \mathcal{M}_{A \leftarrow V} & \mathcal{M}_{A \leftarrow A} \end{pmatrix}, \quad (1)$$

where the block $\mathcal{M}_{X \leftarrow Y}$ specifies whether tokens of modality X may attend to tokens of modality Y . OpenWAM-Infra fixes the two intra-modality blocks: $\mathcal{M}_{V \leftarrow V}$ adopts *first-frame causal* attention, in which noisy frames attend to one another and to the clean first frame while the clean frame attends only to itself, shielding clean conditioning from noise; $\mathcal{M}_{A \leftarrow A}$ adopts *bidirectional* attention, in which the noisy action tokens of a chunk are mutually visible so that the predicted actions inform one another. The two cross-modality blocks then define the four attention mask modes that OpenWAM-Infra supports, as drawn in the right panel of Figure 2: *mutual* enables both $\mathcal{M}_{A \leftarrow V}$ and $\mathcal{M}_{V \leftarrow A}$, so the two modalities attend to each other; *action-sees-video* enables only $\mathcal{M}_{A \leftarrow V}$, letting actions read the predicted world while leaving video generation undisturbed; *video-sees-action* enables only $\mathcal{M}_{V \leftarrow A}$, the reverse; and *isolated* disables both, denoising the two modalities independently. Building on this native support, Section 4.2.2 later compares these modes under controlled settings.

Architectures. The composition rule C specifies where and how information crosses streams, and it does so purely through the execution contract above: it sequences the **prepare**, **per-layer block**, and **finalize** stages of the participating backbones and, when interaction must occur inside attention, splits the block step into its **pre-attention** and **post-attention** halves to substitute the attention computation itself, never modifying backbone internals. Composition rules therefore carry no parameters of their own; all learned capacity resides in the stream backbones. A concrete architecture is a choice $C(\mathcal{E}, \mathcal{S}, \mathcal{M})$; the designs currently supported fall into three families, laid out left to right in the bottom row of Figure 2.

- **Single-System.** This family comprises only the video backbone: the action backbone takes the shared form and injects its tokens into the video sequence, so one transformer processes both modalities with most parameters shared; representative systems include Cosmos Policy (Kim et al., 2026b) and DreamZero (Ye et al., 2026c). OpenWAM-Infra provides two variants, differing in modality-specific capacity:
 - *Vanilla* processes video, action, and proprioceptive tokens as one sequence through the same attention and dense feed-forward blocks, providing no modality-specific capacity.
 - *MoE* retains the shared sequence and self-attention but hard-routes action tokens to a dedicated feed-forward expert while video tokens follow the default path, adding modality-specific capacity without separating the streams (Mu & Lin, 2025).
- **Dual-System.** This family comprises an independent video backbone and action backbone, the latter a dedicated ActionDiT: the two streams hold separate sets of parameters and are connected through self- or cross-attention; representative systems include Fast-WAM (Yuan et al., 2026b) and LingBot-VA (Li et al., 2026b). OpenWAM-Infra provides three variants, differing in how the two streams communicate:
 - *Joint self-attention* merges the hidden states of the two streams into a joint sequence at designated bridge layers, allowing bidirectional token-level interaction before the states return to their streams.
 - *Joint cross-attention* lets the action stream query video features through video-to-action cross-attention at the bridge layers, trained end-to-end so that action-learning gradients update the video stream; optionally, gradients are detached at the video features to isolate action learning from video parameter updates.
 - *IDM* formulates the action module as an inverse-dynamics model conditioned on video features, trained with teacher-forced video states and run in two inference stages: the video trajectory is generated first and the actions are predicted from it.
- **Tri-System.** This family extends the dual layout with a VLM backbone, in which a frozen vision–language model feeds a separate trainable understanding stream; representative systems include Motus (Bi et al., 2026). OpenWAM-Infra provides a single variant:
 - *Joint self-attention* extends the joint sequence to all three streams, which exchange information while retaining stream-specific parameters; the understanding stream joins as a read-only tail that the other streams may attend to while it attends only to itself.

Within each architecture $C(\mathcal{E}, \mathcal{S}, \mathcal{M})$, every module (the visual encoder, the stream backbones, and the visibility attention mask) is instantiated from a registry, orthogonally to the composition rule: every combination is assembled from configuration, and neither the trainer nor the policy server is aware of which architecture is running.

3.2 Training Runtime

Training Formulation. OpenWAM-Infra trains every architecture under one trainer, against one sample contract and one joint flow-matching objective. The trainer never inspects architecture internals: it asks the selected architecture to prepare its own inputs and run its own forward pass, then optimizes the objective on the resulting predictions. Define a sample as $(\ell, \mathbf{o}_{1:T}, \mathbf{a}_{1:H}, \mathbf{q}, \mathbf{m})$, a language instruction, a video window, an action chunk, an optional proprioceptive state, and a per-dimension validity mask. During input preparation, the architecture’s visual encoder $\mathcal{E}$ encodes $\mathbf{o}_{1:T}$ into latents $\mathbf{z}$, and ℓ , optionally joined by $\mathbf{q}$, becomes the context $\mathbf{c}$. Throughout the paper, $t = 0$ denotes pure noise and $t = 1$ clean data. Each stream is noised to its own timestep, t_v for video and t_a for actions, yielding the interpolants $\mathbf{z}^{t_v} = t_v \mathbf{z} + (1 - t_v) \epsilon_v$ and $\mathbf{a}^{t_a} = t_a \mathbf{a} + (1 - t_a) \epsilon_a$.

with Gaussian noise ϵ_v, ϵ_a ; one joint forward pass of the architecture $(\hat{\mathbf{v}}_z, \hat{\mathbf{v}}_a) = \mathbf{v}_\theta(\mathbf{z}^{t_v}, \mathbf{a}^{t_a}, t_v, t_a, \mathbf{c})$ predicts both velocities, and the objective takes the form

$$\mathcal{L} = \lambda_v \mathbb{E}_{t_v, \epsilon_v} \left[w(t_v) \left\| \hat{\mathbf{v}}_z - (\mathbf{z} - \epsilon_v) \right\|_2^2 \right] + \lambda_a \mathbb{E}_{t_a, \epsilon_a} \left[w(t_a) \left\| \mathbf{m} \odot (\hat{\mathbf{v}}_a - (\mathbf{a} - \epsilon_a)) \right\|_2^2 \right], \quad (2)$$

where λ_v, λ_a and $w(\cdot)$ weight the streams and the timesteps, while the validity mask $\mathbf{m}$ restricts the action term to the coordinates an embodiment actually populates, and clean conditioning frames are excluded from the video term. Because t_v and t_a are sampled *independently*, training covers the entire (t_v, t_a) noise plane; any inference schedule, whether it denoises the two streams synchronously at a shared timestep or asynchronously with one stream leading the other, traces a path through this plane and thus remains in-distribution.

Training Utilities. As shown in the Training Utils panel at the center of Figure 2, three core utilities support OpenWAM-Infra training:

1. **Framework.** OpenWAM-Infra integrates DeepSpeed ZeRO (stage 1 or 2) through Accelerate and supports mixed precision (bf16 by default), gradient accumulation, and gradient clipping; a single entry point scales from single-GPU runs to multi-node jobs.
2. **Memory optimization.** To reduce memory consumption, OpenWAM-Infra provides gradient checkpointing, with optional CPU offload of the checkpointed activations, and optimizer-state offload to CPU.
3. **Workflows.** OpenWAM-Infra supports three training workflows. *Pretraining* starts a fresh run. *Fine-tuning* starts a new run initialized from a previous checkpoint: the architecture is rebuilt from the checkpoint’s own record, the new configuration is layered on top, and the identity of the modules the weights belong to is protected from override. *Resume* continues the same run exactly: the full optimizer and scheduler state is restored, training re-enters the data stream at the recorded position, and the run refuses to continue if the dataset’s normalization statistics diverge from those recorded with the run.

Self-Contained Checkpoints. A self-contained checkpoint comprises three parts: the model weights, the fully resolved configuration with every module’s reconstruction specification (and artifacts such as tokenizers) merged in, and the action-normalization statistics. Fine-tuning, resume, and deployment all rebuild the architecture from this record before loading parameters; at deployment, a missing normalization record is a hard error. An evaluation therefore cannot silently change the encoder, the action layout, or the scaling of a trained model, and the checkpoint is exactly what the deployment runtime serves.

3.3 Deployment Runtime

Every checkpoint is served by one policy server, which rebuilds the architecture from its self-contained record and keeps two choices orthogonal: when inference runs (the inference mode) and how the two streams are denoised (the denoising schedule). Figure 3 illustrates the two choices in panels (a) and (b), respectively.

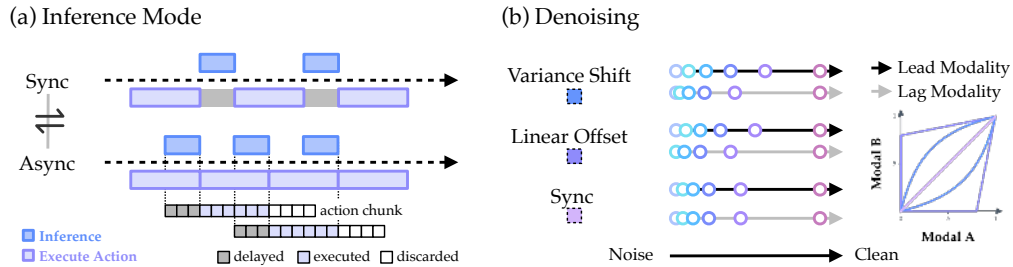


Figure 3 Inference modes and denoising schedules of the deployment runtime. (a) Illustration of the synchronous and asynchronous inference modes. (b) Illustration of the three denoising schedules (variance shift, linear offset, and sync); five denoising steps are drawn for illustration, and circles of the same color denote the timesteps that the two modalities reach at the same denoising step.

Inference Modes. OpenWAM-Infra provides two inference modes over a common buffer mechanism (Figure 3(a)): each inference produces an action chunk that is buffered, and the server pops one action per request. Under *synchronous* inference, the server blocks on a fresh inference whenever the buffer empties, so execution

stalls for the inference latency. Under *asynchronous* inference, let H denote the length of the predicted chunk, $n \leq H$ the inference horizon, and $d < n$ the lead threshold in steps, defaulting to $n/2$. Once only d buffered actions remain, a single background worker prefetches the next chunk while those actions keep executing. The adopted chunk then splits, in order, into a *delayed* prefix of d actions, already covered by the previous buffer while inference ran and therefore skipped; an *executed* window of the next n actions, which becomes the new buffer; and a *discarded* tail of the remaining $\max\{H - d - n, 0\}$ actions. The two modes are indistinguishable to the client: every request is one observation in, one action out.

Denoising Schedules. A denoising schedule is a path $\tau = \{(t_v^i, t_a^i)\}_{i=0}^N$ through the joint noise plane, with $t = 0$ pure noise and $t = 1$ clean data as in Section 3.2; Figure 3(b) draws the three schedules that OpenWAM-Infra supports. Under the *sync* schedule, both streams advance in lockstep along the diagonal: each step performs one joint forward pass and a coupled Euler update in which each stream moves by its own timestep increment; the video latents $\mathbf{z}$ follow $\mathbf{z}^{t_v^{i+1}} = \mathbf{z}^{t_v^i} + (t_v^{i+1} - t_v^i) \hat{\mathbf{v}}_z$, and the action chunk $\mathbf{a}$ follows $\mathbf{a}^{t_a^{i+1}} = \mathbf{a}^{t_a^i} + (t_a^{i+1} - t_a^i) \hat{\mathbf{v}}_a$. Asynchronous schedules let one stream lead through two composable families (Baade et al., 2026),

$$f_\alpha(s) = \frac{\alpha s}{1 + (\alpha - 1)s}, \quad h_o(s) = \max\left\{\frac{s - o}{1 - o}, 0\right\}, \quad (3)$$

where $s = i/N$ denotes global progress, the *variance shift* curve f_α lifts the leading stream above the diagonal for $\alpha > 1$ so that it reaches clean data earlier, and the *linear offset* h_o holds the lagging stream at pure noise until global progress exceeds o . Assigning the lead to the world stream or to the action stream yields the *video-lead* and *action-lead* regimes, and $(\alpha, o) = (1, 0)$ recovers the synchronized diagonal exactly: synchronous serving is a special case rather than a separate code path, and every asynchronous run has an aligned baseline. Because training samples the two timesteps independently (Section 3.2), every such path stays in-distribution. Independently of the schedule shape, each stream’s timestep warp is a backbone property stored in the checkpoint and reused at inference, so the training and serving noise grids cannot drift.

Acceleration. OpenWAM-Infra provides four serving-side accelerations, each independently configurable.

- **Prompt-embedding cache:** a server-lifetime cache maps each prompt to its text-encoder embeddings, removing the text encoder from the per-request path.
- **Video-decode skip:** control consumes actions rather than pixels, so the serving path can skip VAE video decoding entirely.
- **Compilation:** each architecture registers a fixed-shape `torch.compile` path for its inner joint denoising loop, replayed under CUDA graphs to eliminate per-layer launch overhead; the first request carries the compilation warmup.

- **DiT velocity cache:** when the recent velocity predictions of *both* streams are similarity-stable (cosine similarity above a threshold), the next joint forward pass is skipped and the cached velocities are integrated instead, with a bounded number of consecutive skips, following the cross-step reuse of Ye et al. (2026c).

With Wan2.2-TI2V-5B as the video backbone, Figure 4 illustrates the resulting inference speedups across the different architectures.

3.4 Evaluation Protocol

OpenWAM-Infra evaluates trained checkpoints through the policy server of Section 3.3: each benchmark connects as a client, sends observations, and executes the actions returned by the server, as shown in Figure 5.

Client-Server Pipeline. Benchmarks reach the deployment runtime as thin clients over one persistent WebSocket connection and import nothing from the model or training stack. At control step k , the client sends an observation $(\mathbf{o}_k, \ell, \mathbf{q}_k)$: up to three camera views $\mathbf{o}_k$, with the head view required and the wrist views optional; the language instruction ℓ ; and optionally the raw robot state $\mathbf{q}_k$. The response is a single action $\mathbf{a}_k$ in the

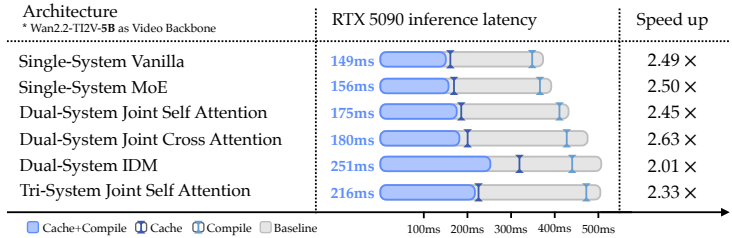


Figure 4 Serving latency across architectures. Inference latency with Wan2.2-TI2V-5B as the video backbone on an RTX 5090. The prompt-embedding cache and video-decode skip are enabled by default; the figure ablates compilation and the DiT velocity cache.

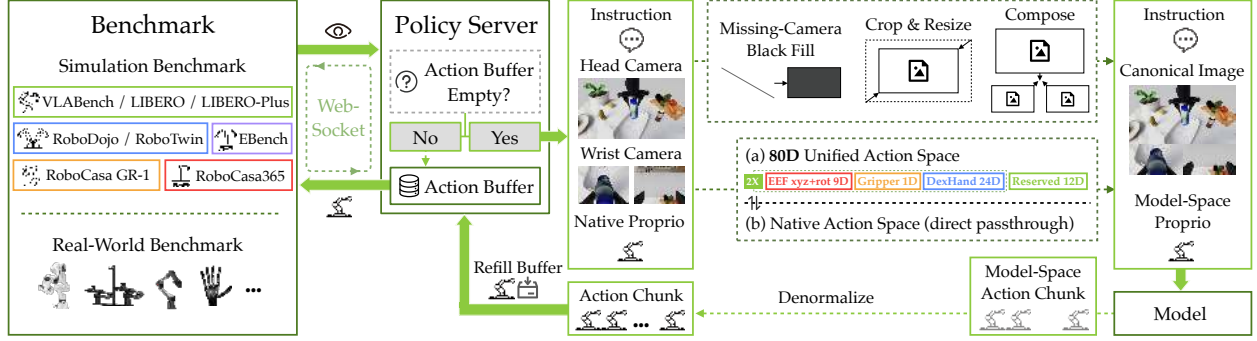


Figure 5 Evaluation protocol of OpenWAM-Infra. Benchmarks connect to the policy server as thin clients over WebSocket. The server canonicalizes each observation, maps the proprioceptive state into the model-side action representation (the 80-D unified action space or the benchmark’s native action space), and denormalizes the predicted action chunk back to native physical units before returning actions.

robot’s native physical units. Every model-facing conversion runs server-side, driven by the self-contained checkpoint of Section 3.2: as laid out in Figure 5, the views are cropped, resized, and composed into the canonical image layout the checkpoint was trained on, with missing cameras filled by black frames, and $\mathbf{q}_k$ is normalized and mapped into the model-side action representation defined below. Inference under the serving stack of Section 3.3 then yields a model-space action chunk $\hat{\mathbf{a}}_{1:H}$, which is mapped back and denormalized into native units before it refills the action buffer from which the server answers requests; normalized values therefore never reach a robot. Between episodes, a single reset request clears all per-episode executor state.

Benchmark Suite. OpenWAM-Infra currently integrates eight simulation benchmarks: LIBERO and LIBERO-Plus (Liu et al., 2023; Fei et al., 2025), VLABench (Zhang et al., 2024), RoboTwin2.0 (Chen et al., 2025), RoboDojo (Chen et al., 2026b), RoboCasa365 (Nasiriany et al., 2026), RoboCasa-GR1 (NVIDIA et al., 2025; Nasiriany et al., 2024), and EBench (Gao et al., 2026), together spanning single-arm and bimanual tabletop manipulation, dexterous-hand humanoid control, and mobile manipulation. Because the protocol exchanges only images, text, and action vectors, real-robot platforms connect through exactly the same interface as the simulators. Each bundled adapter reproduces the observation preprocessing of its benchmark’s training reader, so evaluation-time views match the training distribution; integrating a new benchmark amounts to writing such an adapter, leaving the model and both runtimes untouched.

Action Space Definition. Actions cross this pipeline in one of two representations (Figure 5). By default, every benchmark keeps its *native* action space — RoboTwin2.0, for instance, is served in either a 14-D joint space or a 20-D bimanual end-effector space, and LIBERO in a 10-D end-effector space — so a checkpoint trained on a single benchmark passes actions straight through. Training one model across embodiments, however, requires a single action head over bodies whose native layouts differ in both width and semantics. OpenWAM-Infra therefore also defines a *unified action space* $\mathbf{u} \in \mathbb{R}^{80}$ with fixed slot semantics: two mirrored 34-D arm blocks, each comprising the end-effector position (3), a 6D rotation (6), the gripper (1), and a dexterous hand (24), followed by 12 reserved slots for embodiment-specific channels such as the mobile bases of EBench and RoboCasa365. Since the slot semantics are fixed, the structure that embodiments share lands on the same coordinates. Each dataset declares an index map π from its native dimensions into these slots, with normalization applied *before* scattering and inverted *after* gathering,

$$\mathbf{u} = \text{Scatter}_{\pi}(\text{Norm}(\mathbf{a})), \quad \mathbf{a} = \text{Norm}^{-1}(\text{Gather}_{\pi}(\mathbf{u})), \quad (4)$$

and incoming proprioception traverses the same map in the forward direction. The validity mask $\mathbf{m}$ of Equation (2) marks exactly the mapped slots, so unmapped coordinates receive no gradient during training and remain on their analytic noise path at inference.

4 OpenWAM-Study: Design Principles for World-Action Models

Overview of OpenWAM-Study. Building on the substrate of **OpenWAM-Infra**, we systematically analyze design principles for world-action models through controlled experiments. In this section, we first study how WAMs should inherit upstream world priors in Section 4.1, then understand how to build synergy between the world priors and action learning in Section 4.2, and finally test which design choices generalize to cross-domain embodied pretraining in Section 4.3.

Evaluation Protocol. In this section, we use **RoboTwin2.0** (Chen et al., 2025), a widely-adopted bi-manual manipulation benchmark spanning over 50 tasks, as our evaluation environment. In our experiments, we evaluate under two settings: (1) **In-Domain Performance (RoboTwin2.0-Full)**: Following Bi et al. (2026), we train our model with an entire multi-task data corpus of 2,500 demonstrations collected in clean scenes and 25,000 demonstrations collected under heavy scene randomization, and evaluate under clean and randomized environments separately. (2) **Out-of-Domain Generalization (RoboTwin2.0-Clean2Random)**: Following Yuan et al. (2026a), we train our model on clean data only and evaluate under clean and randomized environments separately. Since the training mixture has never seen randomized scene configurations, it serves as a proxy for evaluating the models’ generalization capabilities to novel scenes. We use success rate as our metric.

4.1 Inheriting Upstream World Knowledge

In this section, we focus on the following question:

Question 1: What world knowledge should WAMs inherit, and how is it best inherited?

World knowledge can be inherited largely through two channels: generative world priors and visual representation priors. Generative world priors refer to the visual and dynamical knowledge embedded in the video generation backbone, whereas visual representation priors refer to the representation space induced by vision encoders (Radford et al., 2021; Caron et al., 2021; Wan et al., 2025).

4.1.1 Generative World Priors

To understand whether and to what extent generative world priors facilitate WAM performance, we compare four video generation backbones with variable parameter counts: Wan2.1-VACE-1.3B (Wan et al., 2025), Cosmos-Predict2.5-2B (Ali et al., 2025), Wan2.2-TI2V-5B (Wan et al., 2025), and Wan2.1-I2V-14B (Wan et al., 2025). We instantiate our World-Action Model with a Dual-System architecture consisting of a video generation module and an action generation module connected with joint self-attention, in which the video generation module parameters are copied from the pretrained video generation model. Results are evaluated on RoboTwin2.0-Full.

Across the four tested backbones, the average success rate of the resulting WAM improves consistently with video generation backbones of increasing capacity (Figure 6). Wan2.1-I2V-14B performs best, while Wan2.2-TI2V-5B trails it by only 1.40 points, even though the former has nearly **3x** the parameter count. Balancing performance against training and deployment efficiency across the four backbones, we ultimately adopt the 5B model as the default for the remaining controlled studies. Since these backbones also differ in architecture, pretraining data, and objective, this comparison establishes backbone choice as a consequential design variable without attributing the entire gain to parameter count alone. The 5B default also keeps subsequent action-side ablations tractable while holding the inherited world prior fixed.

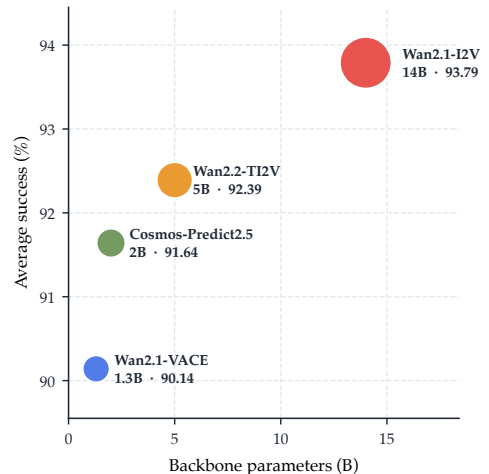


Figure 6 WAM Performance with Different Video Backbone Size. With increasing video generation backbone size, performance of the resulting WAM consistently improves.

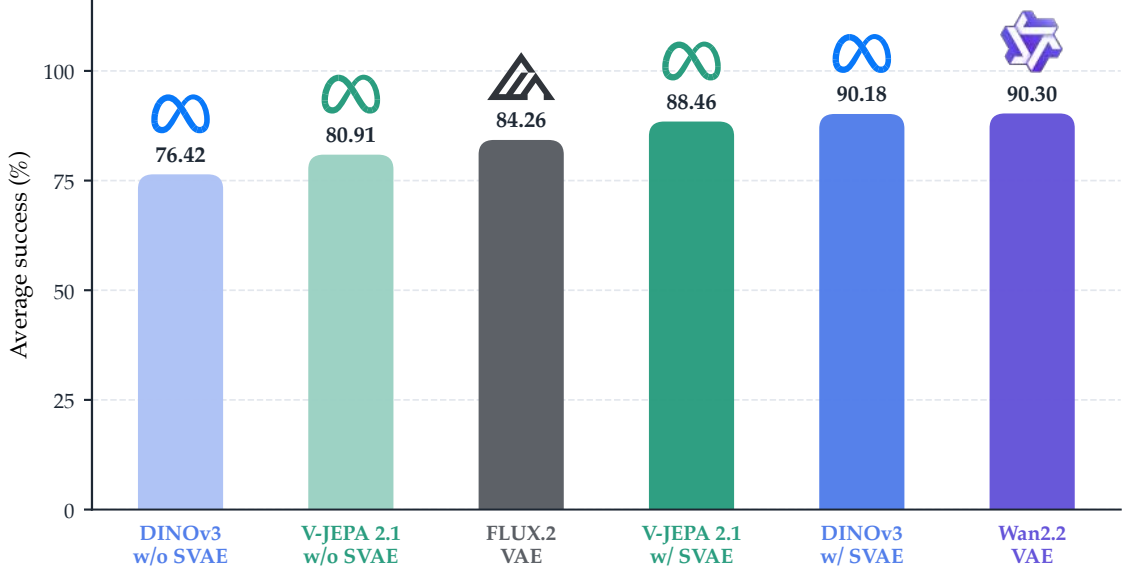


Figure 7 Visual representation priors. We consider building WAMs with both reconstructive and representation encoders, and include a variant of representation encoders with S-VAE (Zhang et al., 2025a), an adapter that converts high-dimensional latents produced by representation encoders into low-dimensional vectors suitable for DiT processing.

4.1.2 Visual Representation Priors

Another important source of world knowledge comes from the latent representation space induced by vision encoders. Broadly speaking, these encoders fall into two categories: (1) **Reconstructive Encoders**: the objective of these encoders is compression, trained solely with pixel-reconstruction; (2) **Representation Encoders**: grounded by self-supervised or multimodal representation learning, these encoders learn semantically structured visual features that provide a basis for visual understanding. Motivated by recent advances in generative and world modeling with representation encoders (Zheng et al., 2026a; Singh et al., 2026; Zhou et al., 2024; Jha et al., 2026; Lyu et al., 2026), we question the design of latent space in the context of world-action models.

In this set of experiments, we choose representative encoders from both categories. For Reconstructive Encoders, we use Wan2.2-VAE (Wan et al., 2025), a state-of-the-art video encoder with a 4x temporal compression rate, and FLUX.2-VAE (Black Forest Labs, 2025), an advanced image encoder yielding highly performant image generation models built on its latent space. For Representation Encoders, we use DINOv3 (Siméoni et al., 2025), the newest generation of DINO (Caron et al., 2021), a classical vision encoder learned through self-supervised learning; and V-JEPA 2.1 (Mur-Labadia et al., 2026), a dense feature encoder based on joint embedding predictive architectures (LeCun et al., 2022). To isolate the performance gain from visual representations alone, we inherit the model architecture of Wan2.2-Ti2V-5B, but randomly initialize its model weights. For a fair comparison, we apply the same 4x temporal compression as Wan2.2-VAE to FLUX.2-VAE, DINOv3, and V-JEPA 2.1: since none of these encoders natively performs temporal compression, we impose the 4x downsampling by averaging the features of every four consecutive frames.

Since modern Diffusion Transformer architectures (Peebles & Xie, 2023) are optimized mostly for reconstructive encoders, naive adoption of representation encoders, which produces high-dimensional latents (e.g., 768-D features for DINOv3 and 1024-D for V-JEPA 2.1), can lead to poor performance due to architectural incompatibility (Zheng et al., 2026a). Following Jha et al. (2026), we include a variant for representation encoders, where we train an S-VAE (Zhang et al., 2025a) adapter that converts the high-dimensional features produced by representation encoders to lower-dimensional vectors (48-D in this experiment, matching the latent dimension of Wan2.2-VAE).

As shown in Figure 7, representation encoders can yield performance on par with or stronger than reconstructive encoders for world-action modeling with the help of S-VAEs. Specifically, while naive adoption of representation encoders yields worse performance than models trained with the reconstructive encoder FLUX.2-VAE, with

dimension contraction using S-VAE, the representation encoders’ contracted variants (DINOv3 w/ SVAE, V-JEPA 2.1 w/ SVAE) significantly outperform FLUX.2-VAE. DINOv3 w/ SVAE achieves nearly on-par performance with Wan2.2-VAE, and we attribute the remaining slim margin to two native advantages of Wan2.2-VAE: it is a reconstructive encoder specifically suited to the video backbone architecture, and its temporal compression is learned natively by the encoder rather than imposed through frame averaging.

Taken together, what determines the quality of a WAM latent space is not the categorical divide between reconstructive and representation encoders, but the operational properties of the latents themselves: compactness (in both the temporal and the token dimension) and rich world information. Priors in representation encoders can thus be inherited to build highly performant WAMs with the help of **temporal compression** and **dimension contraction**. We also encourage active research into building representation encoders with native temporal compression, which in turn may lead to even better prior inheritance.

Finding 1: A WAM inherits upstream world knowledge most effectively through a sufficiently capable generative backbone and a compact, information-rich visual representation space. Reconstructive encoders are not the only option; representation encoders with dimension compression are also performant.

4.2 Building Synergy between World and Action Learning

Inheriting the right priors is not enough; a world–action model needs to build synergy between world and action learning. This requires three decisions at different levels of the system: where action-specific capacity lives, which cross-modal information paths are available during training, and whether inference preserves the noise-state relationship on which those paths were learned.

Question 2: How should inherited world knowledge interact with action learning?

4.2.1 Architectural Capacity

A central question in world–action modeling is how much action-specific capacity a WAM requires and how strongly its video and action streams should be separated. The three architecture families of [Section 3.1](#) span precisely this capacity axis, and we evaluate all six of their variants, instantiating joint cross-attention both end-to-end and with gradients detached at the video features, yielding seven baselines ([Table 1](#)).

Table 1 Architecture Ablation. Averaged success rates (%) on RoboTwin2.0-Full. Bold denotes best values.

Architecture		Success Rate (%)		
System	Variant	Clean	Randomized	Average
Single-System	Vanilla	85.20	85.80	85.50
	MoE	86.22	83.04	84.63
Dual-System	Joint Self-Attention	92.34	92.38	92.36
	Joint Cross-Attention	87.86	88.64	88.25
	Detached Cross-Attention	92.06	91.64	91.85
	IDM	87.76	88.14	87.95
Tri-System	Joint Self-Attention	92.84	92.36	92.60

Results. As shown in [Table 1](#), with increasing architecture capacity, performance from single- to dual- and tri-system continuously improves. Joint self-attention is the strongest dual-system variant, while the tri-system model achieves the best overall performance. Balancing performance with architectural complexity, and isolating the interaction between world knowledge and action learning from the potential influence of the VLM’s understanding features, we therefore adopt dual-system joint self-attention for the remaining experiments, so that the subsequent findings reflect this interaction alone.

4.2.2 Training-Time Information Flow

The architectural comparison selects joint self-attention as the interface between the world and action streams, but joint attention alone does not specify which information flow creates the best synergy. We compare four information flow strategies at training time, controlled by attention masking: **Isolated**, with no cross-stream

communication; **Video Sees Action**, which exposes action features to the world stream; **Action Sees Video**, which exposes world features to the action stream; and **Mutual**, which enables both directions.

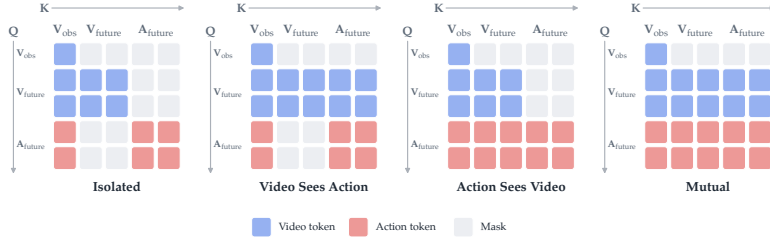


Figure 8 Attention Masking Strategies. We control cross-modality information flow at training time via attention masking.

Table 2 Action learning requires access to world information. Success rates (%) on RoboTwin2.0-Full.

Mask	Success Rate (%)		
	Clean	Random.	Average
Isolated	88.08	86.74	87.41
Video Sees Action	87.92	87.34	87.63
Action Sees Video	92.98	91.80	92.39
Mutual	92.50	91.80	92.15

The comparison separates cleanly according to whether the action stream can access world features (Figure 8 and Table 2). Isolated and video-sees-action masks underperform by roughly five points, whereas action-sees-video and mutual visibility both retain strong performance. World-to-action flow is therefore necessary in this setting. By contrast, adding the reverse action-to-world path changes the from-scratch result only marginally, leaving action-sees-video and mutual visibility as two viable masks to revisit after pretraining.

4.2.3 Inference-Time Information Flow

Training-time attention masking implements *full masking*, while the inference-time denoising schedule provides a softer information gate, i.e. *partial masking*. For this ablation, we fix mutual visibility and train the video and action streams with independently sampled noise levels, covering a two-dimensional space of joint noise states.

We instantiate the schedule abstraction of Section 3.3: synchronized denoising follows the diagonal $(t_v^i, t_a^i) = (s_i, s_i)$, while the variance-shift and linear-offset families of Equation (3) let either stream lead (Baade et al., 2026).

We evaluate $\alpha \in \{4, 8, 16, 32\}$ and $\alpha \in \{0.2, 0.4, 0.6, 0.8\}$ in both leading directions.

Results. Synchronized denoising performs best, and no asynchronous schedule improves performance, regardless of which stream leads or how relative progress is parameterized (Figure 9). With variance-shift schedules, video-leading outperforms action-leading schedules, whereas with linear-offset schedules, action-leading outperforms video-leading schedules. This suggests that while explicit video-to-action information flow is necessary at training time, enforcing such priors through the inference-time denoising schedule does not yield gains, supporting the representation learning hypothesis of world-action modeling (Yuan et al., 2026b) as opposed to an implicit planning-then-IDM schedule at test time (Ye et al., 2026c).

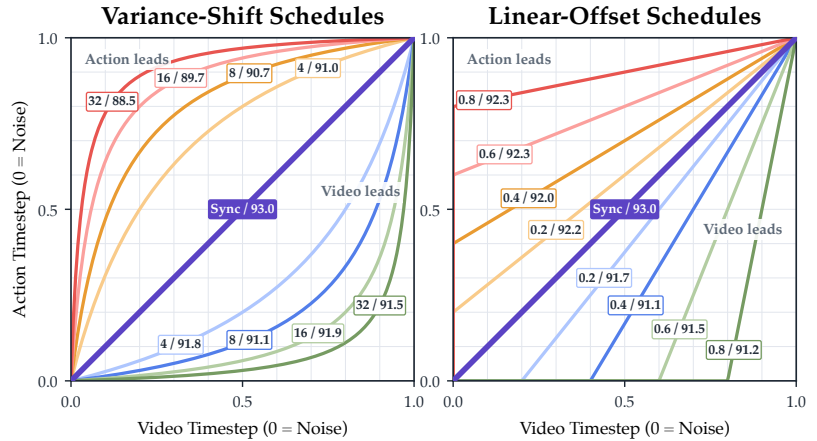


Figure 9 Inference-time Information Flow via Denoising Schedule. Each curve traces action denoising progress against video denoising progress. Curves above/below the diagonal denote action/video first, respectively.

Finding 2: World-action synergy requires explicit world-to-action information flow during training, and synchronized joint denoising at inference. We carry forward dual-system joint self-attention with synchronized denoising and defer the close choice between one-way and mutual visibility to pretraining.

4.3 Consolidating Knowledge Across Domains

The preceding studies identify which world priors to inherit and how world and action streams should interact. However, strong single-domain performance alone does not establish transferable world-action knowledge: the model may simply fit the visual and action distribution of the target tasks, a distinction that cross-domain pretraining sharpens. Robot trajectories provide executable action supervision but limited visual coverage, whereas egocentric video offers broader visual diversity but lacks robot action labels. We therefore ask where pretraining gains arise, how these two sources should be combined, and whether the information-flow choice identified from scratch remains valid after pretraining.

Question 3: How can world-action knowledge be consolidated and transferred across domains?

4.3.1 Problem Setup and Evaluation Protocol

Controlled Transfer Protocol. All runs keep the backbones, optimization budget, and inference procedure fixed — the dual-system joint self-attention architecture with synchronized denoising selected above — and vary only whether and how the model is pretrained with embodiment data; the information-flow mask is revisited in the final ablation. Two complementary protocols serve the evaluation: **RoboTwin2.0-Clean2Random** fine-tunes on Clean and evaluates Clean as in-domain (ID) and Randomized as out-of-domain (OOD), exposing transfer; **RoboTwin2.0-Full** fine-tunes on the full RoboTwin2.0 training set and reports the mean success rate over both conditions.

Embodied Pretraining Data Mixture. We compare supervised fine-tuning from scratch with three pretraining strategies under an identical 600-hour data budget, drawing egocentric human video from EgoDex (Hoque et al., 2025) and real-robot manipulation trajectories from RoboCOIN (Wu et al., 2025). *Robot-only* spends the full 600-hour budget on robot data; the two mixed variants combine 350 hours of egocentric data with 250 hours of robot data, either in two stages (*ego then robot*) or jointly in one stage (*ego + robot co-train*). All four variants then undergo identical downstream fine-tuning.

4.3.2 Embodied Pretraining Primarily Expands OOD Generalization

Pretraining Primarily Improves OOD Generalization. As shown in Figure 10, embodied pretraining yields modest gains for in-domain performance, but yields strong performance gains in OOD evaluation. Embodied pretraining therefore contributes mainly knowledge that transfers beyond the downstream training distribution, rather than better fitting an already saturated ID benchmark.

Robot and Egocentric Data Contribute Different Strengths. Robot-only pretraining yields the strongest ID performance, while both mixed strategies generalize better OOD. This trade-off is consistent with the insight of robot trajectories strengthening executable action grounding and egocentric video broadening the visual and interaction distribution.

Absorbing Egocentric Videos: Sequential versus Co-Training. Sequential training and one-stage co-training performance are nearly matched in both in-domain and OOD evaluation, indicating that using both sources matters more than their precise ordering. Co-training is marginally strongest overall and removes the extra curriculum transition, so we adopt it as the practical default.

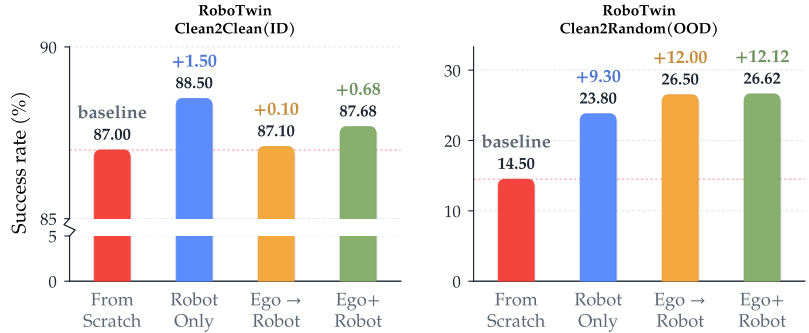


Figure 10 Embodied Pretraining Primarily Improves OOD Generalization. Success rates on RoboTwin2.0-Clean2Random, ordered from lower to higher performance within each evaluation setting.

4.3.3 Pretraining Changes the Preferred Information Flow

The from-scratch ablation establishes that the action stream must see the world stream, but leaves one-way and mutual visibility nearly tied. We repeat this comparison after cross-domain pretraining under both RoboTwin2.0-Clean2Random and RoboTwin2.0-Full.

Mutual Visibility Becomes Preferable with Embodied Pretraining. Without embodied pretraining, RoboTwin2.0-Full slightly favors one-way visibility; after pretraining, however, the same protocol favors Mutual. RoboTwin2.0-Clean2Random shows the same reversal in both ID and OOD, with comparable gains across the two splits (Figure 11). The reversal is therefore neither an artifact of domain shift nor of the evaluation protocol: pretraining turns the world–action interaction into a genuinely bidirectional exchange, in which the predicted future frames provide visual guidance for action generation, while the predicted actions in turn inform the synthesis of the manipulator’s motion in those frames. Without embodied pretraining, data scarcity likely prevents the two streams from reliably establishing such correspondences; the far more abundant pretraining data closes this gap, and Mutual accordingly realizes its advantage once embodied pretraining is in place. We carry Mutual into the final recipe.

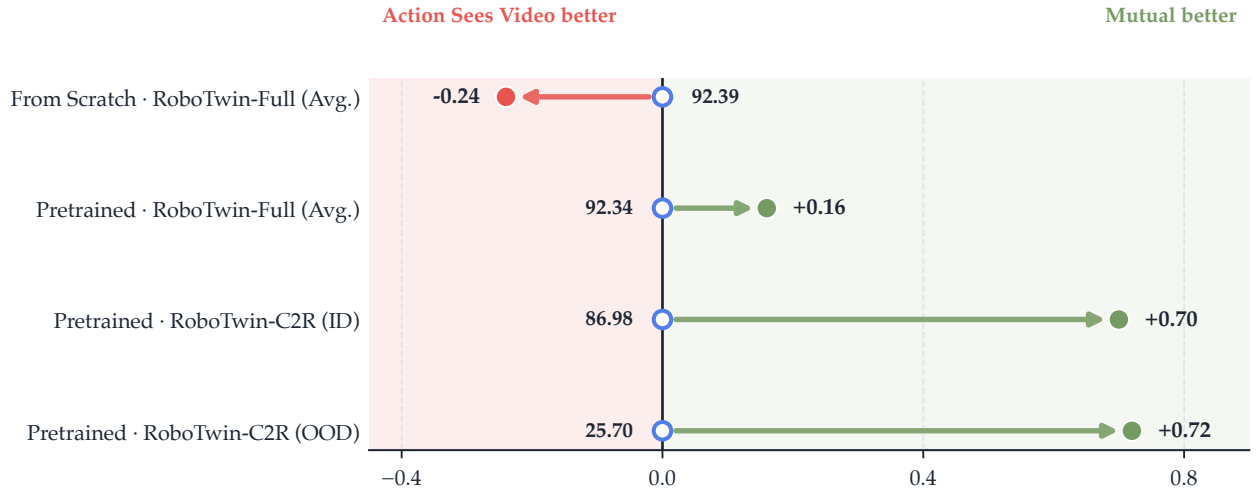


Figure 11 Pretraining Changes the Preferred Information Flow. Central markers give the absolute success rate of Action Sees Video; arrows terminate at the matched Mutual result, with horizontal displacement reporting Mutual – Action Sees Video in percentage points. Green and red denote gains and drops, respectively.

Finding 3: Embodied pretraining primarily expands OOD generalization. Robot trajectories preserve action grounding, egocentric video broadens transfer, and one-stage co-training integrates both effectively. At pretrained scale, mutual world–action visibility is consistently preferred.

4.4 Concluding Remarks

The three questions turn inherited world knowledge into a concrete model design: not a list of individually best hyperparameters, but a sequence in which each decision is tested under the conditions created by the previous one. The next section composes these defaults into **OpenWAM- α** and asks whether they survive full-scale heterogeneous pretraining.

Table 3 The recipe accumulated by OpenWAM-Study. Each evidence-backed choice becomes the default for OpenWAM- α .

Stage	Findings	Carried-forward default
Inherit	Capable video backbones and compact representation latents transfer the strongest upstream priors.	Wan2.2-TI2V-5B; compact latent
Interact	Dedicated action capacity and world-to-action visibility are necessary; synchronized denoising performs best.	Dual joint self-attention; synchronized denoising
Consolidate	Embodied pretraining primarily improves OOD generalization and consistently favors mutual visibility.	One-stage ego + robot co-training; mutual visibility

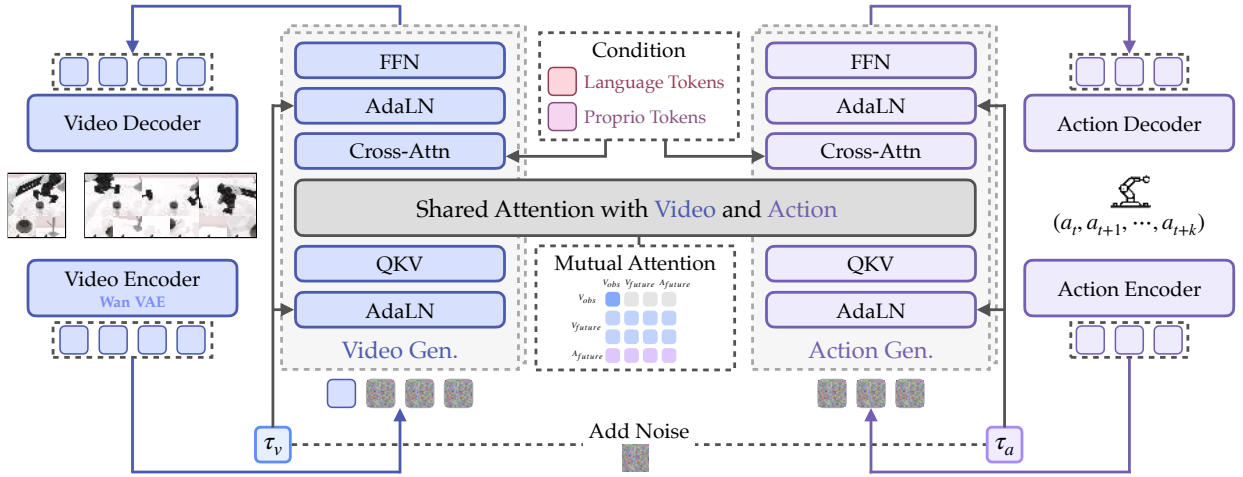
5 OpenWAM- α : From Principles to a Pretrained Model

Overview of OpenWAM- α . Motivated by the design principles and empirical insights uncovered through OpenWAM-Study, we instantiate these findings at scale in OpenWAM- α , an open foundation world-action model for systematically investigating the capabilities and scaling behavior of world-action models across diverse robotic tasks. In Section 5.1, we specify the final architecture, training recipe, and deployment scheme of OpenWAM- α under the guidance of the insights established in Section 4. Section 5.2 then details the pretraining data configuration together with the associated data curation and cleaning pipeline. Finally, Section 5.3 evaluates OpenWAM- α across a diverse set of simulation benchmarks, with analyses of its performance and the key empirical findings revealed by these evaluations, and Section 5.4 further evaluates it on real-world tasks.

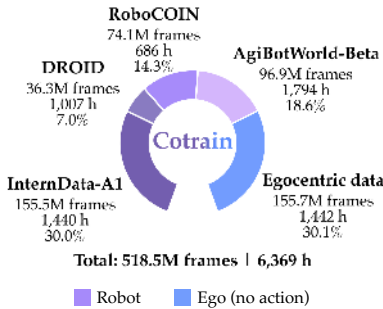
5.1 OpenWAM- α Architecture, Training, and Deployment

As shown in Figure 12, the design principles distilled from OpenWAM-Study determine the configuration of OpenWAM- α across its architecture, training, and deployment stages. The following paragraphs elaborate on each stage in turn.

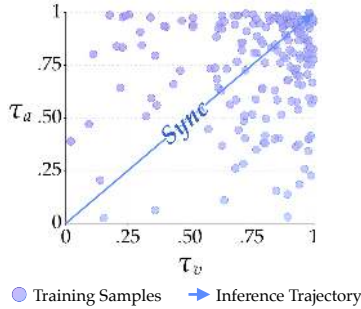
(a) Architecture



(b) Pretraining Data



(c) Scheduler



(d) Action Space

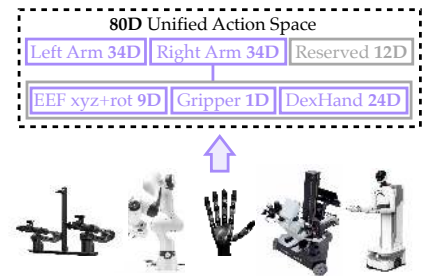


Figure 12 Overview of OpenWAM- α . (a) The dual-system architecture: a video-generation DiT and an ActionDiT jointly denoise the future frames and the action chunk through shared attention under the mutual visibility mask, each conditioned on language and proprioception via cross-attention and carrying its own noise timestep. (b) The pretraining mixture: 518.5M frames (6,369 hours) of egocentric and robot data, co-trained in one stage. (c) Timestep sampling: training covers the full joint noise plane, while inference follows the synchronized diagonal. (d) The 80-D unified action space with fixed slot semantics shared across embodiments.

5.1.1 Architecture

OpenWAM- α adopts the architecture that **OpenWAM-Study** converges to (Table 3), assembled from the modules of Section 3.1 as the composition $C(\mathcal{E}, \mathcal{S}, \mathcal{M})$: the frozen Wan2.2-VAE as the visual encoder $\mathcal{E}$; the pretrained Wan2.2-TI2V-5B DiT (Wan et al., 2025) executing the world stream and a dedicated 1B-parameter ActionDiT executing the action stream, coupled through joint self-attention, as the stream backbones $\mathcal{S} = \{\mathcal{W}, \mathcal{A}\}$; and the *mutual* mode with first-frame-causal intra-video attention as the visibility mask $\mathcal{M}$. Conditioned on the current observation $\mathbf{o}_1$ (all camera views tiled into one canvas), the language instruction ℓ , and the proprioceptive state $\mathbf{q} \in \mathbb{R}^{80}$ expressed in the unified action space, OpenWAM- α jointly denoises the future frames $\mathbf{o}_{2:T}$ of a T -frame video window $\mathbf{o}_{1:T}$ in latent space, together with a continuous action chunk $\mathbf{a} = \mathbf{a}_{1:H} \in \mathbb{R}^{H \times 80}$ (Zhao et al., 2023) (Figure 12a).

Tokenization and Context. The frozen Wan2.2-VAE encodes $\mathbf{o}_{1:T}$ causally – the first frame alone, subsequent frames in groups of four – into $T' = 1 + (T - 1)/4$ latent frames $\mathbf{z}$, so the first latent frame remains a clean anchor of the present. A $(1, 2, 2)$ patch embedding flattens the latent video into world-stream tokens of width 3072 carrying 3D RoPE over the (frame, height, width) grid; each of the H noised action steps is linearly embedded into one action-stream token of width 1024 carrying 1D RoPE over the chunk index. The frozen umT5 encoder maps ℓ into a 4096-dimensional context, a linear projection appends $\mathbf{q}$ as one additional context token, and both streams consume the resulting context $\mathbf{c}$ through their own per-block cross-attention.

Stream Bridging and Prediction. All 30 paired layers of the two backbones act as bridge layers, where stream-owned projections map the two residual widths into a shared attention space of $24 \text{ heads} \times 128 \text{ dimensions}$; under the configured $\mathcal{M}$, the two streams read each other freely while the clean first-frame rows attend to neither noised future frames nor actions. AdaLN injects each stream’s own denoising timestep – t_v token-wise into the world stream, with first-frame tokens pinned to the clean endpoint $t_v = 1$, and t_a into the action stream – and linear heads decode both final states into velocities, realizing the joint forward pass of Section 3.2 at any joint noise state (t_v, t_a) ; how training and inference each cover this plane is specified in Sections 5.1.2 and 5.1.3.

5.1.2 Training

Co-Training Setup. Following the one-stage co-training strategy identified in Section 4.3, OpenWAM- α is trained end to end in a single stage on the ego + robot mixture of Section 5.2. The video DiT (initialized from the pretrained Wan2.2-TI2V-5B weights), the ActionDiT, and the proprioception encoder are all updated; only the umT5 text encoder and the Wan2.2-VAE remain frozen.

Unified Action Supervision. Heterogeneous embodiments meet in the unified action space of Section 3.4: an 80-dimensional vector with fixed slot semantics, comprising two mirrored 34-D arm blocks – end-effector position (3), 6D rotation (6), gripper (1), and dexterous hand (24) – followed by 12 slots reserved for embodiment-specific channels (Figure 12d). Each dataset scatters its native action and state into these slots, the validity mask $\mathbf{m}$ of Equation (2) confines action supervision to the populated coordinates, and the robot state at the start of the window supplies the proprioception token in $\mathbf{c}$.

Objective and Timestep Sampling. OpenWAM- α is trained with the joint flow-matching objective of Equation (2) (Black et al., 2024), instantiated with $\lambda_v = \lambda_a = 1$ and a bell-shaped timestep weight $w(\cdot)$ peaked at intermediate noise levels. The per-stream timesteps are drawn independently as $t_v = 1 - f_\rho(u_v)$ and $t_a = 1 - f_\rho(u_a)$ with $u_v, u_a \sim \mathcal{U}[0, 1]$, where the timestep warp f_ρ of Equation (3) is applied identically to both streams with $\rho = 5$ to bias sampling toward high noise (Figure 12c).

5.1.3 Deployment

Synchronized Denoising. At test time, OpenWAM- α follows the synchronized schedule selected in Section 4.2.3: both streams advance in lockstep along the diagonal of the joint noise plane, realized on the training warp as $t^i = 1 - f_\rho(1 - i/N)$ with $\rho = 5$ and $N = 10$ steps, so neither stream leads. Denoising starts from Gaussian noise – with the first latent frame clamped to the encoding of the current observation and re-pinned after every step – and each step performs one joint forward pass and the coupled Euler update of Section 3.3, so the action chunk is refined against progressively cleaner world features, and vice versa, at all 30 bridge layers. The finished chunk returns to each robot’s native action space through the inverse map of Equation (4).

Inference Mode. All benchmark evaluations that follow, in simulation and in the real world alike, use synchronous inference: whenever the action buffer empties, execution pauses until the model predicts a fresh chunk from the current observation, and the robot then executes that chunk exactly as predicted. The reported results therefore reflect the model’s own performance in the most direct way.

Inference Acceleration. OpenWAM- α is served through the acceleration stack of [Section 3.3](#): the joint denoising loop is compiled into a fixed-shape graph replayed under CUDA graphs, stable velocity predictions are reused across adjacent steps, prompt embeddings are cached, and no VAE decode runs in the control path. The N -step loop completes in roughly 170 ms per chunk on an RTX 5090, well within real-time control budgets.

5.2 Multi-Domain Pretraining Data and Curation

OpenWAM- α is pretrained on multi-domain data drawn from five sources spanning three data types: egocentric human data, real-world robot data, and synthetic robot data. From a raw pool of 1.33B frames ($\approx 14,300$ hours), we construct a training set of 518M frames ($\approx 6,400$ hours) through curation and per-source subsampling ([Table 4](#)). This section describes how the mixture is composed ([Section 5.2.1](#)) and how each source is cleaned ([Section 5.2.2](#)). We further provide the pretraining-stage hyperparameter configuration and other relevant details in [Section B](#).

Table 4 The OpenWAM- α pretraining data. *#Emb.* counts each source’s distinct embodiments. *Task Coverage* marks the manipulation settings each source spans. *Full* reports each source’s raw size before processing, while *Curated + Sampled* reports the data actually used for training, after cleaning ([Section 5.2.2](#)) and per-source whole-episode subsampling ([Section 5.2.1](#)); *Share* is each source’s actual per-epoch sample share under proportional sampling.

Source	Type	#Emb.	FPS	Task Coverage				Full		Curated + Sampled		
				Single	Bimanual	Mobile	Dexterous	Frames (M)	Hours	Frames (M)	Hours	Share (%)
Egocentric data (ours)	Human video	1	30	<i>in-the-wild human manipulation</i>				744.9	6,897	155.7	1,442	30.1
AgiBotWorld-Beta (Bu et al., 2025)	Real robot	1	15		✓	✓	✓	124.5	2,306	96.9	1,794	18.6
RoboCOIN (Wu et al., 2025)	Real robot	15	30		✓	✓	✓	104.5	956	74.1	686	14.3
DROID (Khazatsky et al., 2024)	Real robot	1	10	✓				46.3	1,285	36.3	1,007	7.0
InternData-A1 (Tian et al., 2025)	Simulation	4	30	✓	✓			313.7	2,904	155.5	1,440	30.0
Total		21 robot + human						1,333.9	14,348	518.5	6,369	100.0

5.2.1 Pretraining Data Mixture

Data Composition. Following the co-training recipe of [Section 4.3](#), the mixture combines three complementary data types. Egocentric human data comes from a dataset we carefully constructed for manipulation-centric world modeling – 71.6K long-form first-person recordings of 0.25–6 minutes each, covering 3,006 everyday manipulation tasks; it supplies broad visual and interaction diversity but carries no robot action labels, so its action and proprioception channels remain fully masked and it supervises only the world stream. Real-world robot data (AgiBotWorld-Beta ([Bu et al., 2025](#)), RoboCOIN ([Wu et al., 2025](#)), and DROID ([Khazatsky et al., 2024](#))) grounds the action stream with executable trajectories across 17 physical platforms, while also providing the most faithful visual observations of robots interacting with the physical world. Synthetic robot data (InternData-A1 ([Tian et al., 2025](#))) further broadens the coverage of robot data, encompassing a more comprehensive range of single-arm and bimanual manipulation skills under diverse environmental variations.

Data Budget and Sampling. Considering the compute resources and time cost of pretraining, each source is subsampled under a per-source hour budget. The budgets are derived from frame-based targets – the egocentric and synthetic sources each contribute 30% of the total training frames, and the remaining 40% is divided among the three real-robot sources in proportion to their curated valid-frame counts – so that sources with different native frame rates are balanced by the quantity of data the model actually consumes. Within each source, the hour budget is water-filled across its constituent sub-datasets, and whole episodes are subsampled from the curated pool under a fixed seed until the budget is met. Training then draws samples proportionally to the actual per-source counts, so every retained sample is visited exactly once per epoch.

5.2.2 Data Curation

Aggregating data across different sources and embodiments introduces heterogeneous defects in both the visual and the signal channel. Our cleaning protocol is informed by the data-cleaning pipeline of Qwen-RobotManip (Yuan et al., 2026a), supplemented with rules for the failure modes we observe in the collected sources, and operates at two levels: vision-level cleaning shared by all sources, and signal-level cleaning specific to robot data.

Vision-Level Cleaning. All sources first pass a uniform visual-quality screen that removes undecodable video, frozen or duplicated frames, black, white, and solid-color frames, over- and under-exposure and exposure flicker, blurred frames, and abrupt visual jumps. Egocentric data further exhibits one failure mode of its own: segments in which the hands leave the field of view carry no manipulation signal and are removed; recordings with empty or invalid language annotations are likewise discarded.

Signal-Level Cleaning. Robot data additionally carries state and action channels, which are cleaned in four steps:

1. **Signal integrity.** An episode is discarded outright when its recorded end-effector state fails to track the commanded actions (amplitude ratio $\geq 3\times$ with per-axis correlation < 0.5), or when video-signal misalignment affects more than 2% of frames.
2. **State-first idle detection.** The state channel serves as the primary criterion for idle footage: leading and trailing segments whose state is static – detected with per-robot motion thresholds calibrated from the p99.5 of single-frame deltas, and confirmed when average end-effector translation and geodesic rotation rates fall below 2 cm/s and $5^\circ/\text{s}$ – are trimmed, whereas mid-episode pauses are never cut, since cutting them would splice temporally non-adjacent frames. State discontinuities such as jerk and spike outliers are additionally screened with robust median-MAD thresholds.
3. **Visual cross-checking.** When the state does move, it is cross-checked against the visuals: apparent state motion under which every camera view remains visually static is attributed to sensor jitter and trimmed as well, whereas a genuinely moving arm observed by a frozen camera marks a capture defect and the episode is removed.
4. **Episode-level deletion.** An episode that loses more than 70% of its frames to the steps above, or whose video is frozen for 90% or more of its length, is dropped entirely.

5.3 Simulation Benchmark Evaluation

Starting from the pretrained OpenWAM- α base model, we conduct supervised fine-tuning and evaluation on the eight simulation benchmarks integrated in OpenWAM-Infra (Section 3.4), spanning five embodiment categories:

- **Single-arm:** LIBERO (Liu et al., 2023), LIBERO-Plus (Fei et al., 2025), and VLABench (Zhang et al., 2024);
- **Bimanual:** RoboTwin2.0 (Chen et al., 2025) and RoboDojo (Chen et al., 2026b);
- **Mobile single-arm:** RoboCasa365 (Nasiriany et al., 2026);
- **Mobile bimanual:** EBench (Gao et al., 2026);
- **Dexterous-hand:** RoboCasa-GR1 (NVIDIA et al., 2025; Nasiriany et al., 2024).

For RoboTwin2.0, we evaluate two variants. **RoboTwin2.0-Full** fine-tunes on the mixture of clean and randomized data and then evaluates under both conditions, probing the model’s in-distribution (ID) capability; **RoboTwin2.0-Clean2Random** fine-tunes on clean data only and evaluates under both conditions, probing out-of-distribution (OOD) generalization.

Figure 13 summarizes the scores of OpenWAM- α alongside representative VLA and WAM baselines on each benchmark, with every bar labeled by its actual score, giving a clear account of where the model stands. Figure 14 complements this view by pitting OpenWAM- α against the strongest VLAs and WAMs on every leaderboard, grouped by embodiment, so that the two paradigms can be compared directly. The detailed

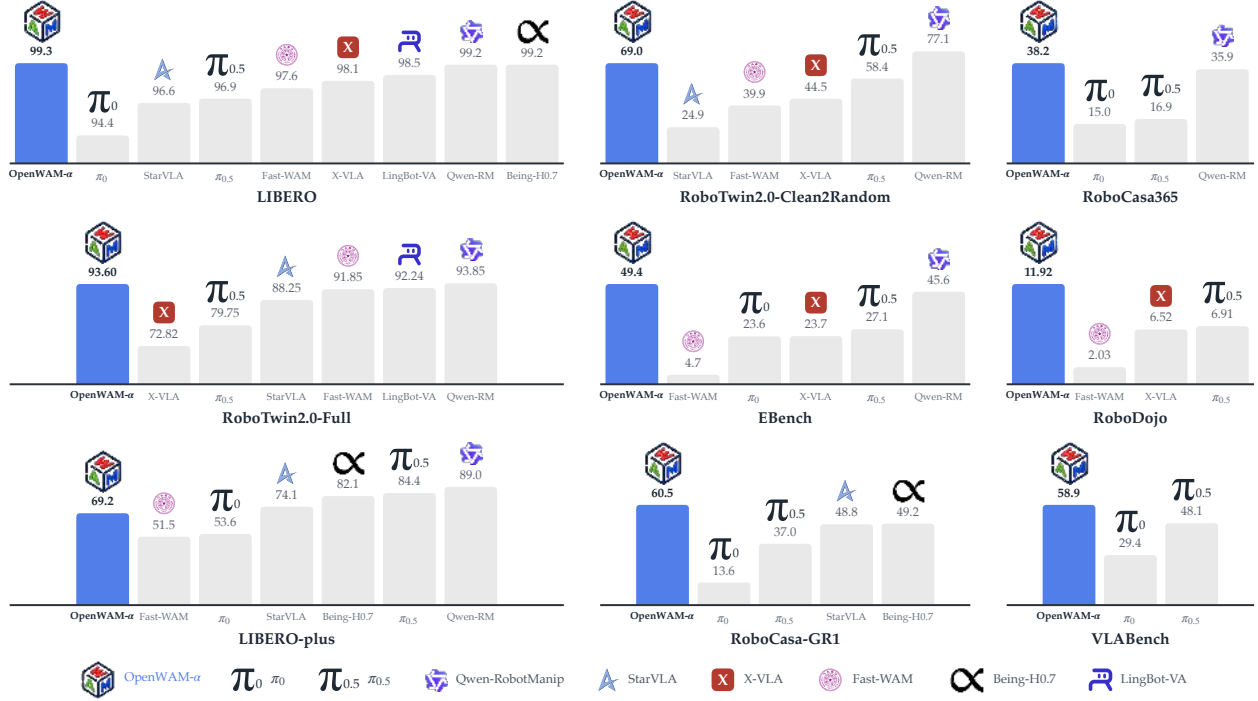


Figure 13 Score comparison of OpenWAM-α against representative VLA and WAM baselines across the simulation benchmarks. Within each panel, baselines are ordered by score, and every bar is labeled with its actual value.

per-benchmark training and evaluation configurations, including hyperparameter settings and evaluation details, are provided in Section B.2. The per-benchmark tables behind both figures are reported in Section D. Through these scores, we seek to answer the two questions at the heart of this evaluation: **(1) how does OpenWAM-α perform, and (2) between VLA and WAM, which paradigm prevails?** The following two subsections address them in turn.

5.3.1 How Does OpenWAM-α Perform?

Across the majority of the benchmarks, OpenWAM-α delivers excellent performance:

- On the single-arm benchmarks **LIBERO** and **VLABench**, the bimanual benchmark **RoboTwin2.0-Full**, the mobile single-arm benchmark **RoboCasa365**, and the dexterous-hand benchmark **RoboCasa-GR1**, OpenWAM-α sits firmly in the top tier, within a marginal gap of the best model.
- On the mobile bimanual benchmark **EBench**, OpenWAM-α sets the state of the art, leading the runner-up QWEN-ROBOTMANIP by roughly 4 points in both SR and Score.
- On the bimanual benchmarks **RoboTwin2.0-Clean2Random** and **RoboDojo**, a gap to the best models (which are VLAs) remains, yet OpenWAM-α is the strongest WAM on both leaderboards, ahead of the other WAMs by a clear margin.

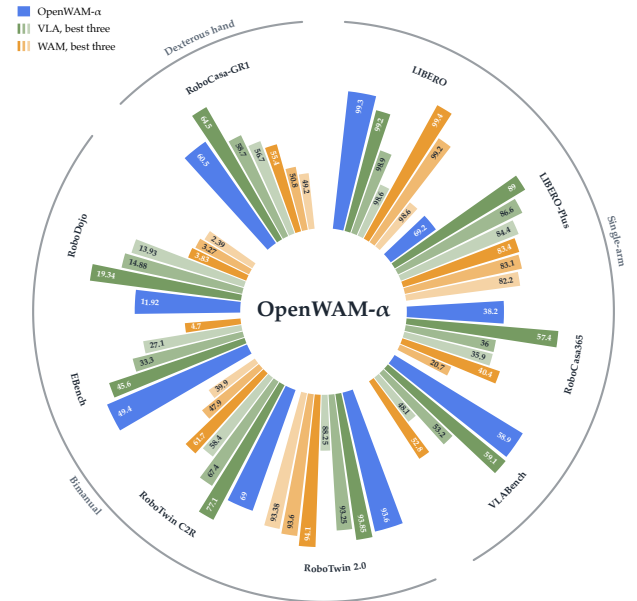


Figure 14 OpenWAM-α against the best of each family, per benchmark, grouped by embodiment.

Table 5 Evaluation Results on LIBERO-Plus. Bold denotes best values, underline second best.

	Camera	Robot	Language	Light	Background	Noise	Layout	Avg
VLA								
π_0 (Black et al., 2024)	13.8	6.0	58.8	85.0	81.4	79.0	68.9	53.6
OpenVLA-OFT (Kim et al., 2025)	56.4	31.9	79.5	88.7	93.3	75.8	74.2	69.6
StarVLA (Community, 2026)	52.5	49.8	88.5	95.7	95.7	73.0	76.9	74.1
ABot-M0 (Yang et al., 2026b)	60.4	67.9	86.4	96.2	91.6	86.4	82.6	80.5
$\pi_{0.5}$ (Physical Intelligence et al., 2025)	78.4	73.6	80.8	96.2	94.1	89.0	84.5	84.4
ACoT-VLA (Zhong et al., 2026)	72.6	<u>82.6</u>	87.5	97.7	<u>96.5</u>	87.8	<u>88.1</u>	<u>86.6</u>
Qwen-RobotManip (Yuan et al., 2026a)	87.2	75.5	85.6	96.6	97.7	97.7	87.3	89.0
WAM								
Fast-WAM (Yuan et al., 2026b)	16.4	44.5	68.9	78.2	53.7	37.7	60.7	51.5
Being-H0.7 (Luo et al., 2026b)	<u>82.0</u>	59.0	82.8	<u>97.8</u>	90.0	93.5	88.5	82.1
Cosmos-Policy (Kim et al., 2026b)	75.8	63.3	81.7	96.5	88.9	92.7	82.2	82.2
ImageWAM (Zhang et al., 2026c)	80.8	50.3	91.4	98.1	85.5	<u>93.8</u>	80.5	83.1
ABot-M0.5 (Chen et al., 2026a)	70.5	87.4	<u>88.6</u>	94.0	89.7	75.5	85.2	83.4
OpenWAM- α	33.8	76.1	88.0	97.0	87.1	39.8	77.5	69.2

The unexpected exception is the single-arm benchmark **LIBERO-Plus**, where the scores of OpenWAM- α fall markedly below its standing elsewhere, as shown in Table 5. The per-perturbation breakdown of **LIBERO-Plus** is telling: the losses concentrate under the *camera* and *noise* perturbations, with visible deficits under the *background* and *layout* perturbations as well. On the very same leaderboard, however, ABot-M0.5, ImageWAM, and Being-H0.7 — all WAMs themselves — perform strongly, with scores approaching the state of the art. We therefore compare OpenWAM- α against these models along two axes, pretraining data and architecture, to identify the underlying causes.

The Data Perspective. Figure 15 contrasts the single-arm portion of the pretraining data of ABot-M0.5 and Being-H0.7 with that of OpenWAM- α . Both baselines pretrain on far larger and more varied single-arm collections, spanning diverse embodiments, scenes, and camera viewpoints, so during pretraining they have already seen visual information and world knowledge close to the LIBERO-Plus test scenes — in viewpoint and noise as much as in background and layout. In contrast, the single-arm data of OpenWAM- α (Table 4) is far smaller in both volume and variety, drawing on only two sources: DROID, collected on a fixed single-arm platform with fixed camera viewpoints, and the single-arm portion of the synthetic InternData-A1. With such limited single-arm coverage, the model receives far less single-arm world knowledge, and its single-arm generalization suffers accordingly on the OOD perturbations of LIBERO-Plus. The converse also holds: the OpenWAM- α mixture is rich in egocentric, bimanual, and dexterous-hand data, and the model is correspondingly strong on the bimanual and dexterous-hand benchmarks.

The Architecture Perspective. Fast-WAM, ABot-M0.5, and OpenWAM- α share one core prediction target — a pixel-level video of the future over a temporal horizon, with the intermediate latents produced by the reconstructive Wan2.2-VAE. On LIBERO-Plus, all three exhibit the same signature: the scores under the *camera* and *noise* perturbations fall clearly below those under the other perturbations, and only ABot-M0.5, backed by its pretraining data, recovers much of the loss relative to Fast-WAM and OpenWAM- α . Pixel-level

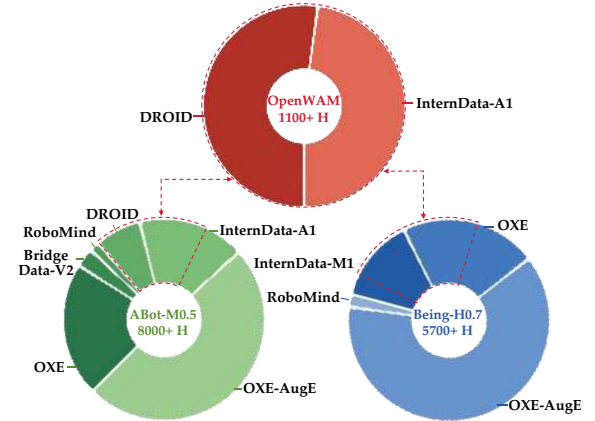


Figure 15 Single-arm pretraining data of ABot-M0.5, Being-H0.7, and OpenWAM- α . The dashed lines indicate that the single-arm data of OpenWAM- α amounts to only a small fraction of what ABot-M0.5 and Being-H0.7 consume.

information is evidently acutely sensitive to camera and noise perturbations — an inherent limitation of pixel-level prediction architectures that only large-scale pretraining can compensate. Two further baselines corroborate this reading. ImageWAM, although not pretrained, remains conspicuously strong on LIBERO-Plus: its prediction target is also a pixel-level latent, but it predicts only a single future frame of the current observation — closer to an edit than a rollout — so no error accumulates across frames and the impact of future-pixel prediction on the camera and noise scores shrinks accordingly. Being-H0.7, in turn, encodes observations with V-JEPA 2.1: its intermediate latents remain temporal (several frames are encoded jointly), yet they are semantic-level features rather than pixel reconstructions, which makes the model markedly more robust to the camera and noise perturbations. Unlike LIBERO-Plus, the OOD designs of the other benchmarks impose no deliberate camera or noise disturbance, so OpenWAM- α remains highly competitive there; on LIBERO-Plus, the camera and noise disturbances compound the single-arm data deficit above, and the performance of OpenWAM- α inevitably degrades.

Takeaway 1: How well an embodied model generalizes on a benchmark is ultimately determined by whether its pretraining mixture contains data close to the benchmark’s test conditions, in both embodiment and environment. Extending [Section 4.3](#), the decisive ingredient of an embodied foundation model remains large-scale, diverse, scene-rich robot manipulation data, which at once covers the broad range of test scenarios a model may later encounter and supplies precise action / visual information grounded in the embodiment — the most direct route to stronger generalization.

Takeaway 2: Driven by large-scale data, WAMs that predict the future in a pixel latent space can achieve excellent performance, yet their robustness to visual disturbance is inherently limited. Echoing [Section 4.1.2](#), a representation that is robust to environmental variation, information-rich, and sufficiently compact is still needed to push WAM performance further.

5.3.2 VLA versus WAM: Which Paradigm Prevails?

Across all benchmarks ([Figure 14](#)), the VLA and WAM groups show no substantial gap in overall success rate, and each side places standout models at the top of leaderboards: ABot-M0.5 and OpenWAM- α among WAMs, Xiaomi-Robotics-1 and QWEN-ROBOTMANIP among VLAs. Since these models differ in pretraining data, architecture, and training configuration alike, neither paradigm can be declared superior outright. The fine-grained scores, however, reveal a consistent pattern: each paradigm holds an advantage region of its own ([Figure 16](#)).

In Distribution, WAMs Fit Better. The cleanest comparison is between StarVLA and Fast-WAM, two models without embodied pretraining ([Figure 16a](#)): on LIBERO, the Clean split of RoboTwin2.0-Clean2Random, and RoboTwin2.0-Full, the WAM attains visibly higher scores on these ID tasks, fitting the training distribution more effectively than its VLA counterpart. The pretrained models tell the same story from both directions ([Figure 16b](#)): OpenWAM- α leads the three strongest VLAs on LIBERO, the In-dist. split of VLABench, and the Clean split of RoboTwin2.0-Clean2Random, and stays within a small gap of the best on the Gen-Std split of RoboDojo — even though the pretraining data of these VLAs exceeds ours. This advantage traces back to the video-latent supervision in WAM training: whereas a VLA is optimized purely against action supervision, with no intermediate latent target, the video-latent term injects an additional source of information into parameter optimization, allowing a WAM to fit the training data more closely.

Out of Distribution, VLAs Generalize Better. The same StarVLA–Fast-WAM comparison reverses out of distribution ([Figure 16a](#)): on LIBERO-Plus the VLA leads by a wide margin, and even on the Randomized split of RoboTwin2.0-Clean2Random, where both models collapse, the ordering still favors the VLA. The pretrained models mirror the reversal ([Figure 16c](#)): on LIBERO-Plus, the Randomized split of RoboTwin2.0-Clean2Random, and the Open split of RoboDojo, OpenWAM- α trails the state-of-the-art VLAs by an evident margin. The mechanism is the flip side of the ID advantage: long-horizon future prediction adds a supervision signal that helps fitting, but under distribution shift the same long horizon means heavier error accumulation — a burden the action-only VLA never carries — so WAMs perform visibly below VLAs in unseen evaluation environments.

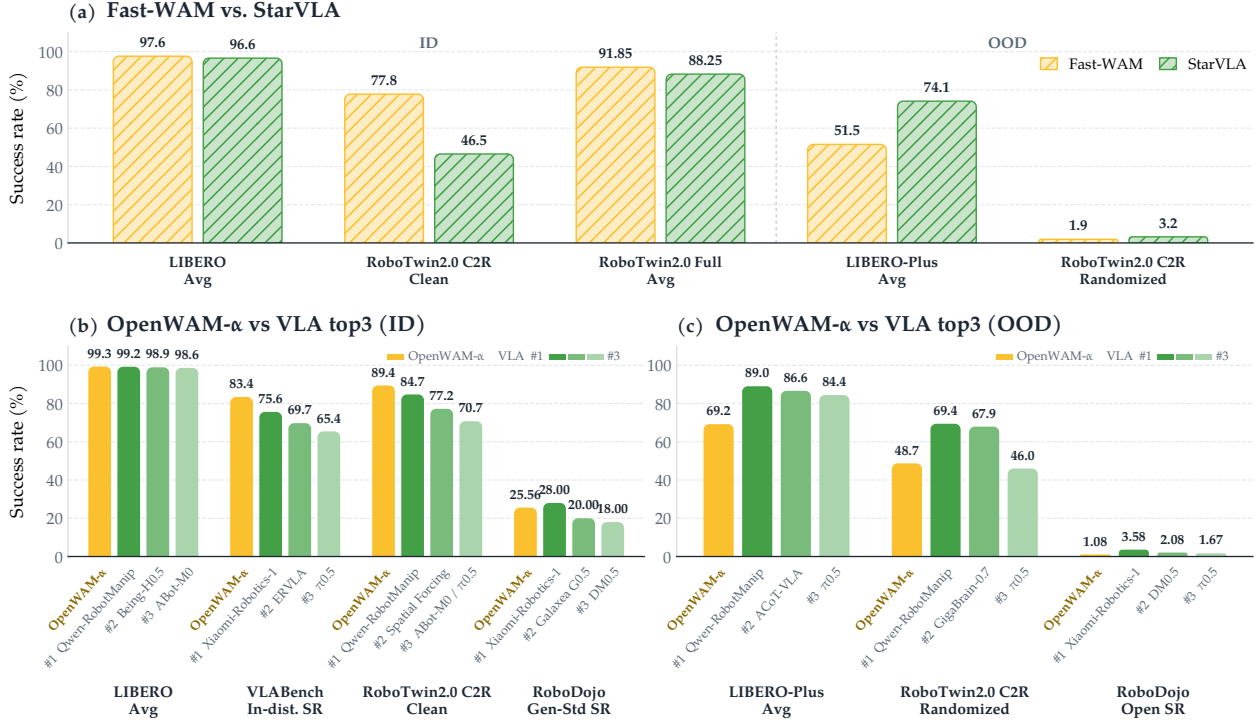


Figure 16 ID and OOD comparisons between the two paradigms. (a) Fast-WAM versus StarVLA, two models without embodied pretraining, on ID and OOD splits. (b) OpenWAM- α versus the three strongest VLAs on ID splits. (c) OpenWAM- α versus the three strongest VLAs on OOD splits.

Crucially, both deficits are remediable by data. Whether it is the ID fitting deficit of VLAs or the OOD generalization deficit of WAMs, sufficiently rich pretraining data — covering complex environmental variation and carrying precise action annotation — lets either paradigm draw on the inherited priors to achieve both strong fitting and strong generalization at test time. Data therefore remains the first priority of model development. At the same time, VLAs and WAMs are both end-to-end models built on the same core information flow, from observation to action; how to combine the complementary strengths of the two paradigms, and thereby push the capability boundary of end-to-end models further, remains a question well worth pursuing.

Takeaway 3: Neither paradigm prevails outright: video-latent supervision gives WAMs the edge in in-distribution fitting, while VLAs generalize better out of distribution — and either deficit can be compensated by sufficiently large and diverse pretraining data. Combining the complementary strengths of the two end-to-end paradigms is a promising route to push the capability boundary further.

5.4 Real-Robot Evaluation

To further examine OpenWAM- α beyond simulation and validate both its general capability and its generalization, we conduct comprehensive real-robot evaluations across three embodiments — single-arm, bimanual, and dexterous-hand — with the experimental setups shown in Figure 17. Specifically:

- **Single-arm experiments** are conducted on the Franka-Research-3 platform and cover three task families — stacking, pick-and-place, and hanging — probing the model’s basic and fine-grained manipulation capabilities. Performance is measured by task success rate (SR).
- **Bimanual experiments** are conducted on the official RoboDojo real-robot platform, spanning three embodiments (ARX X5, Piper, and Piper X); following the RoboDojo task taxonomy, the evaluation covers generalization, precision, long-horizon, memory, and open tasks, assessing the model comprehensively. Performance is measured by SR and Progress Score.

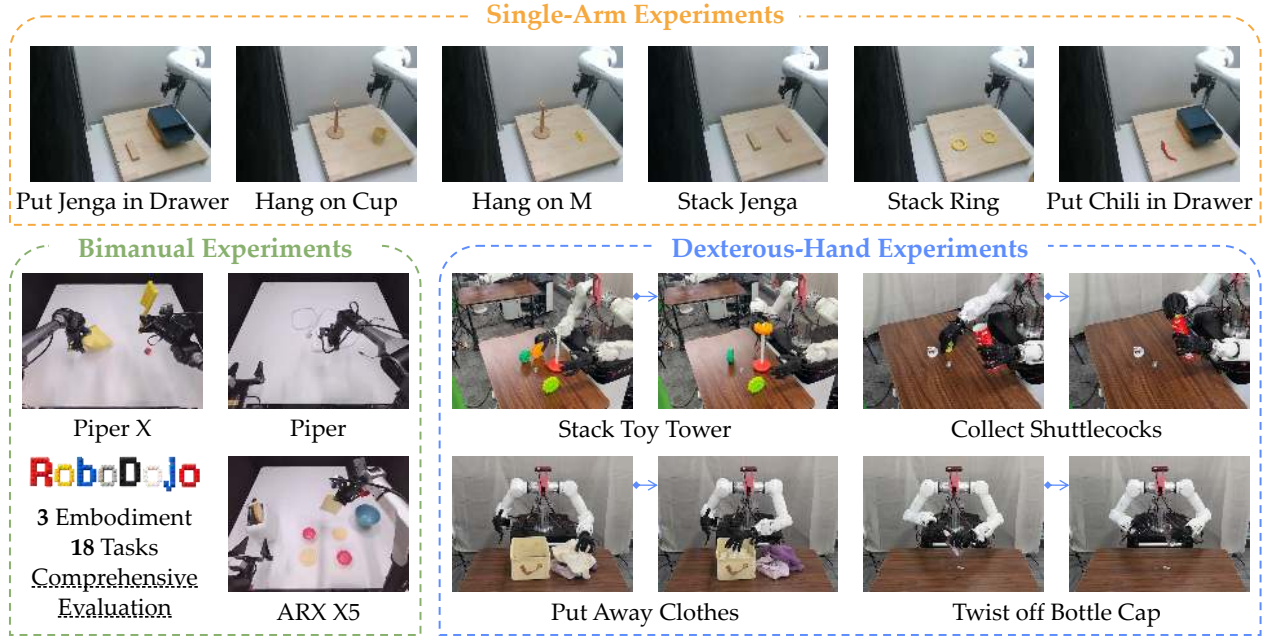


Figure 17 Real-robot experimental setups across three embodiments. *Top:* the six single-arm tasks on the Franka-Research-3 platform. *Bottom left:* the three bimanual embodiments of the RoboDojo real-world track (Piper X, Piper, and ARX X5), covering 18 tasks in total. *Bottom right:* the four dexterous-hand tasks on the Wuji-hand and Tianji-arm platform, each illustrated by key intermediate stages of its execution.

- **Dexterous-hand experiments** are conducted on a platform pairing the Wuji dexterous hand with the Tianji robotic arm — an embodiment and action space absent from the OpenWAM- α pretraining mixture — and cover bimanual-interactive, long-horizon, and fine manipulation tasks, probing how well the model adapts and generalizes to unseen embodiments and unseen action dimensions. Performance is measured by SR and Progress Score.

The task setups and evaluation protocols of each embodiment are documented in [Section C](#), and the SFT configurations used for these experiments are provided in [Section B.2](#).

[Table 6](#), [Table 7](#), and [Table 8](#) report the detailed scores of the single-arm, RoboDojo bimanual, and dexterous-hand experiments, respectively.

Table 6 Evaluation Results on Single-Arm Real-Robot Tasks. Bold denotes best values, underline second best.

	Stack Jenga	Stack Ring	Put Chili in Drawer	Put Jenga in Drawer	Hang on M	Hang on Cup	Avg
$\pi_{0.5}$ (Physical Intelligence et al., 2025)	13/20 (65%)	7/20 (35%)	14/20 (70%)	15/20 (75%)	7/20 (35%)	10/20 (50%)	66/120 (55.0%)
LingBot-VA (Li et al., 2026b)	16/20 (80%)	15/20 (75%)	<u>17/20 (85%)</u>	<u>17/20 (85%)</u>	13/20 (65%)	15/20 (75%)	93/120 (77.5%)
OpenWAM- α	17/20 (85%)	<u>12/20 (60%)</u>	20/20 (100%)	20/20 (100%)	13/20 (65%)	17/20 (85%)	99/120 (82.5%)

On the single-arm platform ([Table 6](#)), OpenWAM- α clearly leads both LingBot-VA, a representative WAM, and $\pi_{0.5}$, a representative VLA, on the majority of tasks, and attains the best average success rate, providing initial evidence of its general and fine-grained manipulation capabilities. We then turn to RoboDojo-Real ([Table 7](#)), a real-robot benchmark that comprehensively evaluates generalist manipulation policies across three bimanual embodiments and a wide range of task dimensions. OpenWAM- α tops the leaderboard: it remains consistently strong across the embodiments and their tasks, and reaches the state of the art on the great majority of them, corroborating the generality and robustness of the model in the real world.

To further probe the extensibility and generalization of OpenWAM- α , we fine-tune the pretrained model on four dexterous manipulation tasks built on the Wuji-hand and Tianji-arm platform and test it under both in-domain and out-of-domain setups ([Table 8](#)). Neither the platform nor its action space — a 9-D

Table 7 Evaluation Results on the Bimanual RoboDojo Real-World Track. Each cell reports Score / SR (%). Bold denotes best values, underline second best. Rows are shaded by embodiment.

Policy	Embodiment	Task 1	Task 2	Task 3	Task 4	Task 5	Task 6	Emb. Avg.	Overall Avg.
X-VLA (Zheng et al., 2026b)	ARX X5	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	2.0 / 0.0	20.7 / 10.0	18.0 / 0.0	6.8 / 1.7	7.6 / 3.3
	Piper	0.0 / 0.0	5.3 / 0.0	0.0 / 0.0	0.0 / 0.0	53.0 / 50.0	37.0 / 0.0	15.9 / 8.3	
	Piper X	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	0.7 / 0.0	0.1 / 0.0	
Xiaomi-Robotics-0 (Cai et al., 2026c)	ARX X5	24.0 / 20.0	0.0 / 0.0	3.0 / 0.0	4.0 / 0.0	40.0 / <u>20.0</u>	19.0 / <u>10.0</u>	15.0 / 8.3	7.9 / 3.9
	Piper	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	23.0 / 20.0	22.7 / 0.0	7.6 / 3.3	
	Piper X	0.0 / 0.0	0.7 / 0.0	<u>4.0</u> / 0.0	0.0 / 0.0	0.0 / 0.0	2.7 / 0.0	1.2 / 0.0	
GalaxeaVLA (G0) (Jiang et al., 2025)	ARX X5	1.0 / 0.0	0.0 / 0.0	3.0 / 0.0	6.0 / 0.0	0.0 / 0.0	10.3 / 0.0	3.4 / 0.0	9.0 / 4.4
	Piper	0.0 / 0.0	<u>32.7</u> / <u>10.0</u>	<u>13.3</u> / <u>10.0</u>	0.0 / 0.0	56.0 / 50.0	30.0 / 10.0	22.0 / 13.3	
	Piper X	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	4.0 / 0.0	0.0 / 0.0	6.0 / 0.0	1.7 / 0.0	
InternVLA-A1 (Cai et al., 2026a)	ARX X5	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	0.0 / 0.0	<u>48.0</u> / <u>20.0</u>	12.0 / 0.0	10.0 / 3.3	12.0 / 7.2
	Piper	0.0 / 0.0	7.3 / 0.0	0.0 / 0.0	0.0 / 0.0	<u>73.0</u> / <u>70.0</u>	59.0 / <u>40.0</u>	23.2 / 18.3	
	Piper X	0.0 / 0.0	0.0 / 0.0	<u>4.0</u> / 0.0	2.0 / 0.0	0.0 / 0.0	<u>10.0</u> / 0.0	2.7 / 0.0	
$\pi_{0.5}$ (Physical Intelligence et al., 2025)	ARX X5	<u>24.6</u> / 20.0	1.8 / 0.0	25.8 / 10.0	<u>47.0</u> / 20.0	40.0 / <u>20.0</u>	<u>26.8</u> / <u>10.0</u>	<u>27.7</u> / <u>13.3</u>	<u>22.9</u> / <u>12.8</u>
	Piper	10.0 / 10.0	28.0 / 0.0	10.0 / <u>10.0</u>	0.0 / 0.0	72.0 / 60.0	<u>72.0</u> / 50.0	<u>32.0</u> / <u>21.7</u>	
	Piper X	0.0 / 0.0	<u>14.8</u> / 10.0	0.0 / 0.0	<u>29.5</u> / <u>10.0</u>	<u>7.5</u> / 0.0	3.0 / 0.0	<u>9.1</u> / <u>3.3</u>	
OpenWAM- α	ARX X5	31.0 / 0.0	0.0 / 0.0	13.0 / 0.0	52.0 / 20.0	100.0 / 100.0	38.0 / 20.0	39.0 / 23.3	37.6 / 24.4
	Piper	<u>8.3</u> / 0.0	60.0 / 40.0	36.7 / 30.0	0.0 / 0.0	100.0 / 100.0	75.0 / 50.0	46.7 / 36.7	
	Piper X	0.0 / 0.0	28.0 / 10.0	18.0 / 0.0	52.3 / 20.0	40.0 / 40.0	24.7 / 10.0	27.2 / 13.3	

Task order. **ARX X5:** cover_blocks, make_bread, make_food, pack_and_pour_fruit, store_in_safe, insert_tubes. **Piper:** stack_and_cover_blocks, fill_pen_holder, put_objects_into_basket, insert_charger, stack_bowls, stand_up_bottles. **Piper X:** classify_objects, disassemble_LEGO, hang_mugs, pack_objects_into_backpack, sweep_blocks, cap_pen.

Table 8 Evaluation Results on Dexterous-Hand Real-Robot Tasks. Each cell reports Score / SR (%); the OOD column aggregates all variation settings of its task. Bold denotes best values.

Method	Stack Toy Tower		Collect Shuttlecocks	
	ID	OOD	ID	OOD
$\pi_{0.5}$	16/30 (53.3) / 1/10 (10%)	12/30 (40.0) / 1/10 (10%)	11/35 (31.4) / 2/10 (20%)	17/54 (31.5) / 3/15 (20%)
OpenWAM- α	21/30 (70.0) / 4/10 (40%)	17/30 (56.7) / 3/10 (30%)	29/35 (82.9) / 6/10 (60%)	30/57 (52.6) / 4/15 (26.7%)

Method	Put Away Clothes		Twist off Bottle Cap	
	ID	OOD	ID	OOD
$\pi_{0.5}$	26/30 (86.7) / 6/10 (60%)	45/60 (75.0) / 9/20 (45%)	8/10 (80.0) / 3/10 (30%)	13/20 (65.0) / 6/20 (30%)
OpenWAM- α	30/30 (100.0) / 10/10 (100%)	55/60 (91.7) / 17/20 (85%)	10/10 (100.0) / 7/10 (70%)	18/20 (90.0) / 16/20 (80%)

end-effector pose combined with 21 dexterous-hand degrees of freedom — ever appears in the OpenWAM- α pretraining mixture; nevertheless, OpenWAM- α outperforms $\pi_{0.5}$ by a clear margin across all tasks and setups, demonstrating that the model adapts reliably and stably to an entirely unseen embodiment.

Together, these experiments assess OpenWAM- α across three embodiment types and a broad spectrum of real-world manipulation tasks. The results show that OpenWAM- α performs strongly in every setting and stands on par with today’s leading models, indicating that it can serve as a strong baseline for further development and comparison by the community.

6 Conclusions

This work introduced **OpenWAM**, an open research stack that turns world-action modeling from a set of tightly coupled implementation choices into a controlled experimental program. **OpenWAM-Infra** factorizes the WAM design space into composable modules assembled into three architecture families, served by a single trainer, policy server, and evaluation protocol spanning eight simulation benchmarks and real robots. On this substrate, **OpenWAM-Study** examined what world knowledge a WAM should inherit, how world and action learning create synergy, and how that synergy consolidates across domains, distilling the answers into a concrete recipe. **OpenWAM- α** then instantiated this recipe at scale on egocentric human and robot data through a unified action space, delivering consistently strong results across the simulation benchmarks and real-robot experiments on single-arm, bimanual, and dexterous-hand platforms.

We release the full stack, including the infrastructure, evaluation protocols, pretrained weights, and data recipes, as a shared and reproducible foundation for world-action research, with OpenWAM- α serving as a strong baseline for further development and comparison. Looking ahead, these findings point to larger and more diverse embodied data with precise action annotation, visual representations that are both compact and robust to environmental variation, and end-to-end designs that combine the complementary strengths of WAMs and VLAs as the most promising directions for WAMs. A more detailed discussion of limitations and future work is provided in [Section A](#).

Acknowledgements

We thank Nilaksh and Chuning Zhu for their helpful discussions. We thank Wuji Technology for providing compute resources, which are crucial for the completion of this project.

References

- [1] Ali, A., Bai, J., Bala, M., Balaji, Y., Blakeman, A., Cai, T., Cao, J., Cao, T., Cha, E., Chao, Y.-W., et al. World simulation with video foundation models for physical ai. *arXiv preprint arXiv:2511.00062*, 2025.
- [2] Allen-Zhu, Z. Physics of language models: Part 4.1, architecture design and the magic of canon layers. *Advances in Neural Information Processing Systems*, 38:42349–42369, 2026.
- [3] Baade, A., Chan, E. R., Sargent, K., Chen, C., Johnson, J., Adeli, E., and Fei-Fei, L. Latent forcing: Reordering the diffusion trajectory for pixel-space image generation. *arXiv preprint arXiv:2602.11401*, 2026.
- [4] Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., Ge, W., Guo, Z., Huang, Q., Huang, J., Huang, F., Hui, B., Jiang, S., Li, Z., Li, M., Li, M., Li, K., Lin, Z., Lin, J., Liu, X., Liu, J., Liu, C., Liu, Y., Liu, D., Liu, S., Lu, D., Luo, R., Lv, C., Men, R., Meng, L., Ren, X., Ren, X., Song, S., Sun, Y., Tang, J., Tu, J., Wan, J., Wang, P., Wang, P., Wang, Q., Wang, Y., Xie, T., Xu, Y., Xu, H., Xu, J., Yang, Z., Yang, M., Yang, J., Yang, A., Yu, B., Zhang, F., Zhang, H., Zhang, X., Zheng, B., Zhong, H., Zhou, J., Zhou, F., Zhou, J., Zhu, Y., and Zhu, K. Qwen3-vl technical report, 2025. URL <https://arxiv.org/abs/2511.21631>.
- [5] Bai, Y., Wang, H., Dai, M., Zhong, Q., Liu, Y., and Lin, L. Bridge-wa: Predicting where and how the world changes for robotic action. *arXiv preprint arXiv:2607.02195*, 2026.
- [6] Barreiros, J., Beaulieu, A., Bhat, A., Cory, R., Cousineau, E., Dai, H., Fang, C.-H., Hashimoto, K., Irshad, M. Z., Itkina, M., et al. A careful examination of large behavior models for multitask dexterous manipulation. *Science Robotics*, 11(113):eaea6201, 2026.
- [7] Beyer, L., Steiner, A., Pinto, A. S., Kolesnikov, A., Wang, X., Salz, D., Neumann, M., Alabdulmohsin, I., Tschannen, M., Bugliarello, E., et al. Paligemma: A versatile 3b vlm for transfer. *arXiv preprint arXiv:2407.07726*, 2024.
- [8] Bi, H., Tan, H., Xie, S., Wang, Z., Huang, S., Liu, H., Zhao, R., Feng, Y., Xiang, C., Rong, Y., et al. Motus: A unified latent action world model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 35101–35113, 2026.
- [9] Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [10] Black Forest Labs. FLUX.2: Analyzing and enhancing the latent space of FLUX. Technical blog, 2025. URL <https://bfl.ai/research/representation-comparison>.
- [11] Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Choromanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- [12] Brooks, T., Peebles, B., Holmes, C., DePue, W., Guo, Y., Jing, L., Schnurr, D., Taylor, J., Luhman, T., Luhman, E., Ng, C., Wang, R., and Ramesh, A. Video generation models as world simulators, 2024. URL <https://openai.com/research/video-generation-models-as-world-simulators>.
- [13] Bu, Q., Cai, J., Chen, L., Cui, X., Ding, Y., Feng, S., Gao, S., He, X., Huang, X., Jiang, S., et al. Agibot world colosseum: A large-scale manipulation platform for scalable and intelligent embodied systems. *arXiv preprint arXiv:2503.06669*, 2025.

- [14] Cai, J., Cai, Z., Cao, J., Chen, Y., He, Z., Jiang, L., Li, H., Li, H., Li, Y., Liu, Y., et al. Internvla-a1: Unifying understanding, generation and action for robotic manipulation. *arXiv preprint arXiv:2601.02456*, 2026a.
- [15] Cai, J., Ling, L., Chu, S., Liu, Z., Kang, J., Liang, Z., Xu, W., Mao, Y., Zhang, W., Yang, X., et al. Aha-wam: Asynchronous horizon-adaptive world-action modeling with observation-guided context routing. *arXiv preprint arXiv:2606.09811*, 2026b.
- [16] Cai, R., Guo, J., He, X., Jin, P., Li, J., Lin, B., Liu, F., Liu, W., Ma, F., Ma, K., et al. Xiaomi-robotics-0: An open-sourced vision-language-action model with real-time execution. *arXiv preprint arXiv:2602.12684*, 2026c.
- [17] Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., and Joulin, A. Emerging properties in self-supervised vision transformers. In *2021 IEEE/CVF international conference on computer vision (ICCV)*, pp. 9630–9640. IEEE, 2021.
- [18] Chen, R., Yang, Y., Tang, Z., Huo, D., Lin, T., Wu, H., Liu, H., Chen, Y., Zheng, L., Yuan, B., et al. Abot-m0.5: Unified mobility-and-manipulation world action model. *arXiv preprint arXiv:2607.00678*, 2026a.
- [19] Chen, T., Chen, Z., Chen, B., Cai, Z., Liu, Y., Li, Z., Liang, Q., Lin, X., Ge, Y., Gu, Z., et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.
- [20] Chen, T., Chen, Y., Li, Z., Tang, J., Su, K., Lu, H., Wan, W., Chen, B., Liu, S., Yan, H., Su, H., Dou, Z., Wang, K., Zhang, D., Liu, Y., Qin, Y., Liang, Q., Wu, Q., Lin, Z., Lin, W., Wang, Y., He, M., Wu, T., Wu, R., Zhou, J., Lei, K.-C., Yu, H., Ji, Y., Jin, W., Lin, G., Li, X., Xiong, Q., Xu, R., Li, Z., Chai, W., Xie, E., Wang, Z., Mu, Y., Dong, H., Matusik, W., Ding, M., Ding, W., Luo, P., and Tomizuka, M. Robodojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies, 2026b. URL <https://arxiv.org/abs/2607.04434>.
- [21] Chi, C., Xu, Z., Pan, C., Cousineau, E., Burchfiel, B., Feng, S., Tedrake, R., and Song, S. Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots. *arXiv preprint arXiv:2402.10329*, 2024.
- [22] Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., and Song, S. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- [23] Community, S. Starvla: A lego-like codebase for vision-language-action model developing. *arXiv preprint arXiv:2604.05014*, 2026.
- [24] Community, X., Chen, T., Chen, Y., Nian, T., Cai, Z., Chen, G., Lin, W., Liang, Q., Xiang, P., Su, K., et al. XPolicyLab: A unified standard and open ecosystem for robot policy evaluation and deployment. *arXiv preprint arXiv:2608.09892*, 2026.
- [25] Dexmal. DM0.5. Technical blog, 2026. URL <https://www.dexmal.com/blog/dm0.5>.
- [26] Fei, S., Wang, S., Shi, J., Dai, Z., Cai, J., Qian, P., Ji, L., He, X., Zhang, S., Fei, Z., Fu, J., Gong, J., and Qiu, X. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- [27] Gao, N., Zheng, J., Gao, X., Ma, H., Wang, H., Wang, Y., Chen, J., Chen, Z., Zhang, S., Jia, M., Jiang, X., Zhu, Z., Li, X., Wang, S., Li, H., Cai, W., Yang, Y., Xu, X., Lyu, Z., Mu, Y., Wang, T., Pang, J., Zeng, J., Zhang, W., and Shen, C. Ebench: Elemental diagnosis of generalist mobile manipulation policies. *arXiv preprint arXiv:2606.18239*, 2026.
- [28] Guo, J., Li, Q., Li, P., Chen, Z., Sun, N., Su, Y., Wang, H., Zhang, Y., Li, X., and Liu, H. Unified 4d world action modeling from video priors with asynchronous denoising. *arXiv preprint arXiv:2604.26694*, 2026.
- [29] Ha, D. and Schmidhuber, J. World models. *arXiv preprint arXiv:1803.10122*, 2(3):440, 2018.
- [30] Hafner, D., Lillicrap, T., Ba, J., and Norouzi, M. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.
- [31] Han, J., Tong, S., Fan, D., Chen, M., Torr, P., Kokkinos, F., and Lewis, M. Towards physics of multimodal pretraining: Knowledge flow, modality synergy, early unification, and recipes. *arXiv preprint arXiv:2608.05000*, 2026.
- [32] Ho, J., Salimans, T., Gritsenko, A., Chan, W., Norouzi, M., and Fleet, D. J. Video diffusion models. *Advances in neural information processing systems*, 35:8633–8646, 2022.
- [33] Hoque, R., Huang, P., Yoon, D. J., Sivapurapu, M., and Zhang, J. Egodex: Learning dexterous manipulation from large-scale egocentric video. *arXiv preprint arXiv:2505.11709*, 2025.

- [34] Hu, Y., Zhu, H., Zheng, B., Hu, Y., Zhang, T., Chen, Z., Zhao, J., Nai, R., and Gao, Y. Openhlm: An empirical recipe for whole-body humanoid loco-manipulation. *arXiv preprint arXiv:2606.22174*, 2026.
- [35] Huang, S., Wu, J., Zhou, Q., Miao, S., and Long, M. Vid2world: Crafting video diffusion models to interactive world models. *arXiv preprint arXiv:2505.14357*, 2025.
- [36] Huang, S., Kaushik, P., Chen, M., Pan, H., Geng, K., Chehab, O., Moreno-Pino, F., and Simchowitz, M. Nano world models: A minimalist implementation of future video prediction. *arXiv preprint arXiv:2605.23993*, 2026.
- [37] Jha, S., Zholus, A., Chandar, S., et al. Reconstruction or semantics? what makes a latent space useful for robotic world models. *arXiv preprint arXiv:2605.06388*, 2026.
- [38] Jiang, T., Yuan, T., Liu, Y., Lu, C., Cui, J., Liu, X., Cheng, S., Gao, J., Xu, H., and Zhao, H. Galaxea open-world dataset and g0 dual-system vla model. *arXiv preprint arXiv:2509.00576*, 2025.
- [39] Karras, T., Aittala, M., Aila, T., and Laine, S. Elucidating the design space of diffusion-based generative models. *Advances in neural information processing systems*, 35:26565–26577, 2022.
- [40] Khazatsky, A., Pertsch, K., Nair, S., Balakrishna, A., Dasari, S., Karamcheti, S., Nasiriany, S., Srirama, M. K., Chen, L. Y., Ellis, K., et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.
- [41] Kim, D., Jang, H., Koo, M., Jang, S., Kim, T., Kim, B., Yoon, B., Jang, C., Choi, D., Han, D., et al. Rldx-1 technical report. *arXiv preprint arXiv:2605.03269*, 2026a.
- [42] Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [43] Kim, M. J., Finn, C., and Liang, P. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [44] Kim, M. J., Gao, Y., Lin, T.-Y., Lin, Y.-C., Ge, Y., Lam, G., Liang, P., Song, S., Liu, M.-Y., Finn, C., et al. Cosmos policy: Fine-tuning video models for visuomotor control and planning. *arXiv preprint arXiv:2601.16163*, 2026b.
- [45] Lazzati, F., Stachowicz, K., Chen, W., Metelli, A. M., Wagenmaker, A., and Levine, S. Why does action chunking improve behavioral cloning performance in robotic control? *arXiv preprint arXiv:2608.02547*, 2026.
- [46] LeCun, Y. et al. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62(1): 1–62, 2022.
- [47] Li, F., Song, W., Zhao, H., Wang, J., Ding, P., Wang, D., Zeng, L., and Li, H. Spatial forcing: Implicit spatial representation alignment for vision-language-action model. In *International Conference on Learning Representations*, 2026a.
- [48] Li, L., Zhang, Q., Luo, Y., Yang, S., Wang, R., Han, F., Yu, M., Gao, Z., Xue, N., Zhu, X., et al. Causal world modeling for robot control. *arXiv preprint arXiv:2601.21998*, 2026b.
- [49] Lin, F., Arora, K., Mercat, J., Nishimura, H., Shah, P., Xu, C., Zhang, M., Zolotas, M., Angeles, M., Pfannenstiehl, O., et al. A systematic study of data modalities and strategies for co-training large behavior models for robot manipulation. *arXiv preprint arXiv:2602.01067*, 2026a.
- [50] Lin, X., Lian, S., Yu, B., Yang, R., Shen, Z., Wu, C., Miao, Y., Jin, Y., Shi, Y., He, J., Huang, C., Cheng, B., and Chen, K. Physbrain: Human egocentric data as a bridge from vision language models to physical intelligence, 2026b. URL <https://arxiv.org/abs/2512.16793>.
- [51] Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., and Stone, P. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.
- [52] Liu, I., Cheng, A.-C., Yan, R., Chen, G., Qiu, R.-Z., Zou, X., Yi, S., Yin, H., Wang, X., and Liu, S. Long-horizon manipulation via trace-conditioned vla planning. *arXiv preprint arXiv:2604.21924*, 2026a.
- [53] Liu, Y., Dong, Z., Ye, B., Yuan, T., Jiang, T., Yang, A., Cao, S., Liu, H., Sun, Y., Guo, Z., et al. G0.5: One autoregressive stream for robot reasoning and action. *arXiv preprint arXiv:2608.11739*, 2026b.
- [54] Liu, Z., Mao, H., Wu, C., Feichtenhofer, C., Darrell, T., and Xie, S. A convnet for the 2020s. *CoRR*, abs/2201.03545, 2022. URL <https://arxiv.org/abs/2201.03545>.

- [55] Luo, H., Wang, Y., Zhang, W., Zheng, S., Xi, Z., Xu, C., Xu, H., Yuan, H., Zhang, C., Wang, Y., et al. Being-h0.5: Scaling human-centric robot learning for cross-embodiment generalization. *arXiv preprint arXiv:2601.12993*, 2026a.
- [56] Luo, H., Zhang, W., Feng, Y., Zheng, S., Xu, H., Xu, C., Xi, Z., Fu, Y., and Lu, Z. Being-h0.7: A latent world-action model from egocentric videos. *arXiv preprint arXiv:2605.00078*, 2026b.
- [57] Lyu, J., Liu, K., Zhang, X., Liao, H., Feng, Y., Zhu, W., Shen, T., Chen, J., Zhang, J., Dong, Y., et al. Lda-1b: Scaling latent dynamics action model via universal embodied data ingestion. *arXiv preprint arXiv:2602.12215*, 2026.
- [58] M. Moerland, T., Broekens, J., Plaat, A., and M. Jonker, C. Model-based reinforcement learning: A survey. *Foundations and Trends in Machine Learning*, 16(1):1–118, 2023.
- [59] Ma, T., Zheng, J., Wang, Z., Jiang, C., Cui, A., Liang, J., and Yang, S. Dit4dit: Jointly modeling video dynamics and actions for generalizable robot control. *arXiv preprint arXiv:2603.10448*, 2026.
- [60] Maes, L., Lidec, Q. L., Scieur, D., LeCun, Y., and Balestrieri, R. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.
- [61] McKinzie, B., Gan, Z., Fauconnier, J.-P., Dodge, S., Zhang, B., Dufter, P., Shah, D., Du, X., Peng, F., Belyi, A., et al. Mm1: methods, analysis and insights from multimodal llm pre-training. In *European Conference on Computer Vision*, pp. 304–323. Springer, 2024.
- [62] Mu, S. and Lin, S. A comprehensive survey of mixture-of-experts: Algorithms, theory, and applications. *arXiv preprint arXiv:2503.07137*, 2025.
- [63] Mur-Labadia, L., Muckley, M., Bar, A., Assran, M., Sinha, K., Rabbat, M., LeCun, Y., Ballas, N., and Bardes, A. V-jepa 2.1: Unlocking dense features in video self-supervised learning. *arXiv preprint arXiv:2603.14482*, 2026.
- [64] Nasiriany, S., Maddukuri, A., Zhang, L., Parikh, A., Lo, A., Joshi, A., Mandlekar, A., and Zhu, Y. Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.
- [65] Nasiriany, S., Nasiriany, S., Maddukuri, A., and Zhu, Y. Robocasa365: A large-scale simulation framework for training and benchmarking generalist robots. In *International Conference on Learning Representations (ICLR)*, 2026.
- [66] NVIDIA, Bjorck, J., Castañeda, F., Cherniadev, N., et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [67] Pai, J., Achenbach, L., Montesinos, V., Forrai, B., Mees, O., and Nava, E. mimic-video: Video-action models for generalizable robot control beyond vlas. *arXiv preprint arXiv:2512.15692*, 2025.
- [68] Pan, C., Anantharaman, G., Huang, N.-C., Jin, C., Pfrommer, D., Yuan, C., Permenter, F., Qu, G., Boffi, N., Shi, G., et al. Much ado about noising: Dispelling the myths of generative robotic control. In *International Conference on Learning Representations*, volume 2026, pp. 90575–90614, 2026.
- [69] Peebles, W. and Xie, S. Scalable diffusion models with transformers. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 4172–4182. IEEE, 2023.
- [70] Physical Intelligence, Black, K., Brown, N., Darpinian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [71] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pp. 8748–8763. PmLR, 2021.
- [72] Simchowitz, M., Pfrommer, D., and Jadbabaie, A. The pitfalls of imitation learning when actions are continuous. *arXiv preprint arXiv:2503.09722*, 2025.
- [73] Siméoni, O., Vo, H. V., Seitzer, M., Baldassarre, F., Oquab, M., et al. Dinov3. *arXiv preprint arXiv:2508.10104*, 2025.
- [74] Singh, J., Zheng, B., Wu, Z., Zhang, R., Shechtman, E., and Xie, S. Improved baselines with representation autoencoders. *arXiv preprint arXiv:2605.18324*, 2026.
- [75] Sun, N., Zhang, Y., Yang, Y., Zhao, W., Li, P., Guo, J., Song, W., Ding, P., Suo, R., Su, Y., et al. Revisiting embodied chain-of-thought for generalizable robot manipulation. *arXiv preprint arXiv:2606.03784*, 2026.

- [76] Team, G., Ye, A., Sun, A., Jin, C., Cheng, C., Shi, C., Shang, D., Zhang, D., Huang, G., Wang, G., et al. Gigabrain-0.7: Scaling embodied foundation models to emergent capabilities with a three-system architecture. *arXiv preprint arXiv:2608.15875*, 2026a.
- [77] Team, X. R., Guo, J., Jin, P., Li, J., Li, P., Li, Y., Liu, F., Peng, W., Qin, O., Su, Y., et al. Xiaomi-robotics-1: Scaling vision-language-action models with over 100k hours of real-world trajectories. *arXiv preprint arXiv:2607.15330*, 2026b.
- [78] Tian, Y., Yang, Y., Xie, Y., Cai, Z., Shi, X., Gao, N., Liu, H., Jiang, X., Qiu, Z., Yuan, F., et al. Interndata-a1: Pioneering high-fidelity synthetic data for pre-training generalist policy. *arXiv preprint arXiv:2511.16651*, 2025.
- [79] Tong, S., Brown, E., Wu, P., Woo, S., Middepogu, M., Akula, S. C., Yang, J., Yang, S., Iyer, A., Pan, X., et al. Cambrian-1: A fully open, vision-centric exploration of multimodal llms. *Advances in Neural Information Processing Systems*, 37:87310–87356, 2024.
- [80] Tong, S., Fan, D., Nguyen, J., Brown, E., Zhou, G., Qian, S., Zheng, B., Vallaes, T., Han, J., Fergus, R., et al. Beyond language modeling: An exploration of multimodal pretraining. *arXiv preprint arXiv:2603.03276*, 2026.
- [81] Wan, T., Wang, A., Ai, B., Wen, B., Mao, C., Xie, C.-W., Chen, D., Yu, F., Zhao, H., Yang, J., et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.
- [82] Wang, Q., Li, M., Guan, J., Ye, J., Xie, S., Liu, Y., Chen, J., Liang, Z., Zhang, J., Hu, X., et al. Qwen-vla: Unifying vision-language-action modeling across tasks, environments, and robot embodiments. *arXiv preprint arXiv:2605.30280*, 2026a.
- [83] Wang, Y. *Instructions for Practical Living, and Other Neo-Confucian Writing*. Columbia University Press, New York, 1963.
- [84] Wang, Y., Syed, R., Wu, F., Zhang, M., Onol, A., Barreiros, J., Nayyeri, H., Dear, T., Zhang, H., and Li, Y. Interactive world simulator for robot policy training and evaluation. *arXiv preprint arXiv:2603.08546*, 2026b.
- [85] Wang, Z., Chen, Y., Liu, Y., Ye, J., Chen, P., Lu, C., Liu, S., Yu, B., and Jia, J. Vp-vla: Visual prompting as an interface for vision-language-action models. *arXiv preprint arXiv:2603.22003*, 2026c.
- [86] Wen, K., Hall, D., Ma, T., and Liang, P. Fantastic pretraining optimizers and where to find them. In *International Conference on Learning Representations*, volume 2026, pp. 144731–144838, 2026.
- [87] Wu, S., Liu, X., Xie, S., Wang, P., Li, X., Yang, B., Li, Z., Zhu, K., Wu, H., Liu, Y., et al. Robocoin: An open-sourced bimanual robotic data collection for integrated manipulation. *arXiv preprint arXiv:2511.17441*, 2025.
- [88] Xu, M., Zhang, H., Hou, Y., Xu, Z., Fan, L., Veloso, M., and Song, S. Dexumi: Using human hand as the universal manipulation interface for dexterous manipulation. In *Conference on Robot Learning*, pp. 437–459. PMLR, 2025.
- [89] Yang, L., Song, W., Wang, X., Sheng, P., Fang, Z., Zhou, Z., He, J., Yan, H., Chen, J., Sun, N., Sun, Q., Wang, P., Liu, L., Wang, Y., Gao, Y., Dayoub, F., and Li, H. 4d-wam: Infusing spatiotemporal awareness into world action models through trajectory fields, 2026a. URL <https://arxiv.org/abs/2608.08023>.
- [90] Yang, Y., Zeng, S., Lin, T., Chang, X., Qi, D., Xiao, J., Liu, H., Chen, R., Chen, Y., Huo, D., et al. Abot-m0: Vla foundation model for robotic manipulation with action manifold learning. *arXiv preprint arXiv:2602.11236*, 2026b.
- [91] Ye, A., Wang, B., Ni, C., Huang, G., Zhao, G., Li, H., Li, H., Li, J., Lv, J., Liu, J., et al. Gigaworld-policy: An efficient action-centered world-action model. *arXiv preprint arXiv:2603.17240*, 2026a.
- [92] Ye, J., Gao, N., Yang, S., Zheng, J., Wang, Z., Chen, Y., Chen, P., Chen, Y., Liu, S., and Jia, J. Starvla- α : Reducing complexity in vision-language-action systems. *arXiv preprint arXiv:2604.11757*, 2026b.
- [93] Ye, S., Ge, Y., Zheng, K., Gao, S., Yu, S., Kurian, G., Indupuru, S., Tan, Y. L., Zhu, C., Xiang, J., et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026c.
- [94] Ye, Y., Fu, Y., Lv, Y., Hou, B., Cen, J., Kong, L., Zheng, D., Chen, T., Liu, J., Cao, Z., et al. Data pyramid for embodied manipulation. *arXiv preprint arXiv:2607.24744*, 2026d.
- [95] Yuan, H., Liang, Z., Chen, A., Wang, Y., Li, H., Lin, P., Huang, Y., Lei, Z., Zhang, T., Zhang, J., et al. Qwen-robotmanip technical report: Alignment unlocks scale for robotic manipulation foundation models. *arXiv preprint arXiv:2606.17846*, 2026a.
- [96] Yuan, T., Dong, Z., Liu, Y., and Zhao, H. Fast-wam: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026b.

- [97] Zhang, H., Xiang, L., Lin, H., Huang, Z., Wang, M., Zhong, D., Dong, Y., Wu, Y., Rao, Y., Zhang, D., et al. Hy-embodied-0.5-vla: From vision-language-action models to a real-world robot learning stack. *arXiv preprint arXiv:2606.14409*, 2026a.
- [98] Zhang, Q., Li, L., Zhang, L., Yang, S., Luo, Y., Li, S., Wang, R., Wang, J., Shao, J., Xu, G., et al. Native video-action pretraining for generalizable robot control. *arXiv preprint arXiv:2607.08639*, 2026b.
- [99] Zhang, S., Xu, Z., Liu, P., Yu, X., Li, Y., Gao, Q., Fei, Z., Yin, Z., Wu, Z., Jiang, Y.-G., and Qiu, X. Vlabench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks. *arXiv preprint arXiv:2412.18194*, 2024.
- [100] Zhang, S., Zhang, H., Zhang, Z., Ge, C., Xue, S., Liu, S., Ren, M., Kim, S. Y., Zhou, Y., Liu, Q., et al. Both semantics and reconstruction matter: Making representation encoders ready for text-to-image generation and editing. *arXiv preprint arXiv:2512.17909*, 2025a.
- [101] Zhang, T. T., Pfrommer, D., Pan, C., Matni, N., and Simchowitz, M. Action chunking and exploratory data collection yield exponential improvements in behavior cloning for continuous control. *arXiv preprint arXiv:2507.09061*, 2025b.
- [102] Zhang, Y., Zhang, W., Qi, Z., Zhang, H., Lin, H., Zhang, J., Mu, Y., Yang, X., Zeng, W., and Jin, X. Imagewam: Do world action models really need video generation, or just image editing? *arXiv preprint arXiv:2606.19531*, 2026c.
- [103] Zhao, T. Z., Kumar, V., Levine, S., and Finn, C. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- [104] Zheng, B., Ma, N., Tong, S., and Xie, S. Diffusion transformers with representation autoencoders. In *International Conference on Learning Representations*, volume 2026, pp. 35791–35820, 2026a.
- [105] Zheng, J., Li, J., Wang, Z., Liu, D., Kang, X., Feng, Y., Zheng, Y., Zou, J., Chen, Y., Zeng, J., et al. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *International Conference on Learning Representations*, 2026b.
- [106] Zhong, L., Liu, Y., Wei, Y., Xiong, Z., Liu, S., and Ren, G. Acot-vla: Action chain-of-thought for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 8152–8162, 2026.
- [107] Zhou, G., Pan, H., LeCun, Y., and Pinto, L. Dino-wm: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.
- [108] Zhu, C., Yu, R., Feng, S., Burchfiel, B., Shah, P., and Gupta, A. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. *arXiv preprint arXiv:2504.02792*, 2025.

Appendix

This appendix provides supplementary analyses and implementation details supporting the main paper:

- §A discusses limitations of our work and directions for future research.
- §B documents the pretraining configuration and the dataset-specific SFT configurations used during post-training.
- §C details the task setups and evaluation protocols of the real-world experiments.
- §D reports the full per-benchmark simulation scores behind Figure 13.

A Limitations and Future Work

While OpenWAM provides a fully-open, systematic exploration towards world–action model pretraining, it has several limitations and opens up interesting future directions worth exploring.

1. **Training phases.** We mostly focus on the embodied pretraining phase of world–action modeling. Post-training and adaptation methods can lead to significant improvements for embodied foundation models, and the empirical recipe as well as underlying mechanisms for these methods remain open questions.
2. **Architecture.** Across the six architecture variants currently supported by OpenWAM, we mostly explore modality fusion through cross-modality attention or hard-routed MoE. Drawing experience from the Unified Multimodal Model (UMM) community, we encourage future work to explore more native modality-fusion techniques, such as tokenization-phase early fusion and soft-routed MoE.
3. **Pretraining data mixture.** We did not include UMI-style (e.g., UMI (Chi et al., 2024), DexUMI (Xu et al., 2025)) collected data. In theory, UMI-style data offers task and scene diversity comparable to human egocentric videos, which is a crucial component for out-of-domain generalization capabilities. Co-training with data that contain robot-executable actions but are diverse in scene and task level, which can either be collected through UMI-style interfaces or post-processing pipelines, may offer a more data-efficient path towards autonomous embodied machine intelligence. In addition, we look forward to further breakthroughs in simulation for embodied AI: simulation can natively generate robot manipulation data with diverse scenes and realistic motion trajectories, unconstrained by the time and labor costs of the physical world, and thus holds unbounded potential for scaling robot data by orders of magnitude.
4. **Visual encoder.** Weighing the compression of candidate encoders in both the temporal and the token dimension, we ultimately adopt Wan2.2-VAE as the final encoder of OpenWAM — a choice that reflects the best trade-off currently available rather than an optimal solution: our evaluations reveal that pixel-reconstruction encoders such as Wan2.2-VAE are not sufficiently robust to viewpoint, noise, and scene variations. A latent representation that is compact while carrying sufficient environment information is still needed to push WAM performance further, and merits deeper exploration.

B Training Details

This section documents the optimization and data-loading configurations used to train and adapt OpenWAM- α . We organize the details into two stages: multi-domain pretraining and dataset-specific supervised fine-tuning (SFT) during post-training.

B.1 Pretraining Configuration

The key hyperparameters used for multi-domain pretraining are summarized in Table 9. Pretraining uses 16 nodes with eight NVIDIA H200 GPUs per node (128 GPUs in total) and takes approximately seven days. With 24 clips per GPU and no gradient accumulation, the global batch size is 3,072 clips per optimizer step. The source-level data budgets and realized mixture proportions are reported separately in Table 4.

Table 9 Pretraining configuration for OpenWAM- α .

Configuration	Value
Compute	16 nodes (128 NVIDIA H200 GPUs)
Training time	≈ 7 days
Optimizer	AdamW
Batch size	3,072 (24 per GPU)
Learning rate	1×10^{-4}
LR schedule	Cosine; 5% warmup; minimum ratio 0.01
Weight decay	0.01
Optimizer momentum	$\beta_1, \beta_2 = 0.9, 0.95$
Training iterations	155,862 (1 epoch)
Gradient clipping	Global norm 1.0
Model precision	bfloat16
Distributed training	DeepSpeed ZeRO Stage 2
Input clip	33 frames; video stride 4; window stride 1
Image resolution	384×320
Multi-view input	Enabled
Image augmentation	ColorJitter(0.2, 0.2, 0.2, 0.0)
Flow shifts	Video/action: 5.0/5.0
Loss weights	$\lambda_v = 1.0, \lambda_a = 1.0$
Unified control space	80-D action; 80-D proprioceptive state

B.2 Dataset-Specific SFT Configuration

All downstream models are initialized from the same pretrained OpenWAM- α checkpoint. Supervised fine-tuning keeps the pretraining configuration of Table 9 unchanged and differs only in the three benchmark-dependent settings summarized in Table 10: the global batch size, the number of training epochs or steps, and whether image augmentation is applied. Training length is given in epochs over the fine-tuning set, with the corresponding number of optimizer steps in parentheses, or directly in optimizer steps where no epoch-based schedule was used. Image augmentation, where enabled, is the same ColorJitter(0.2, 0.2, 0.2, 0.0) used in pretraining. LIBERO-Plus is evaluated with the LIBERO checkpoint without further fine-tuning.

Table 10 Dataset-specific SFT configuration for OpenWAM- α . Settings not listed are identical to pretraining (Table 9); “–” denotes no image augmentation.

Benchmark	Batch size	Training Epochs / Steps	Augmentation
Simulation benchmarks			
LIBERO	256	10 epochs (10,690 steps)	–
VLABench	196	6k steps	ColorJitter
RoboTwin2.0-Full	256	5 epochs (118,655 steps)	–
RoboTwin2.0-Clean2Random	256	5 epochs (10,740 steps)	ColorJitter
RoboDojo	256	60k steps	ColorJitter
RoboCasa365	1,024	60k steps	ColorJitter
EBench	256	100k steps	ColorJitter
RoboCasa-GR1	256	100k steps	ColorJitter
Real-robot experiments			
Single-arm (Franka-Research-3)	256	10 epochs (9,860 steps)	–
Bimanual (RoboDojo real-world track)	256	30k steps	ColorJitter
Dexterous hand (Wuji + Tianji)	256	5 epochs (10,925 steps)	–

C Real-World Evaluation Protocols

This section details the real-world evaluation of [Section 5.4](#): for each embodiment, we document the task setup and the corresponding evaluation protocol.

C.1 Single-Arm Real-Robot Experiments

Task Setup. We evaluate single-arm policies on the Franka-Research-3 platform using six real-world tabletop tasks, covering stacking, hanging, and drawer manipulation. The tasks use a Franka-Research-3 arm with a parallel gripper and RGB observation cameras, as shown in [Figure 18](#). Each task is specified by a natural-language instruction and instantiated with a fixed physical scene: stacking tasks place two target objects on the tabletop, hanging tasks place the object and shelf in the workspace, and drawer tasks place the object on the table next to an upper drawer.

Table 11 Task instructions used in the single-arm real-robot evaluation.

Task	Instruction
Stack Ring	Pick the yellow ring on the left side, stack it on the other ring.
Stack Jenga	Pick the jenga on the left side, stack it on the other jenga.
Hang on Cup	Pick the cup on the table, hang it on the shelf.
Hang on M	Pick the M-shaped object on the table, hang it on the shelf.
Put Chili in Drawer	Pick the chili on the table, put it into the drawer, then push the upper drawer closed.
Put Jenga in Drawer	Pick up the jenga block on the table, put it into the drawer, then push the upper drawer closed.



Figure 18 Single-arm real-robot setup on the Franka-Research-3 platform. The workspace contains the drawer, stacking objects, and hanging fixtures used across the six tasks, while the robot is equipped with an Intel RealSense camera and a Robotiq parallel gripper for closed-loop execution.

The corresponding execution sequences are visualized in [Figure 19](#), where each row contains six frames uniformly sampled from one rollout video of the task.

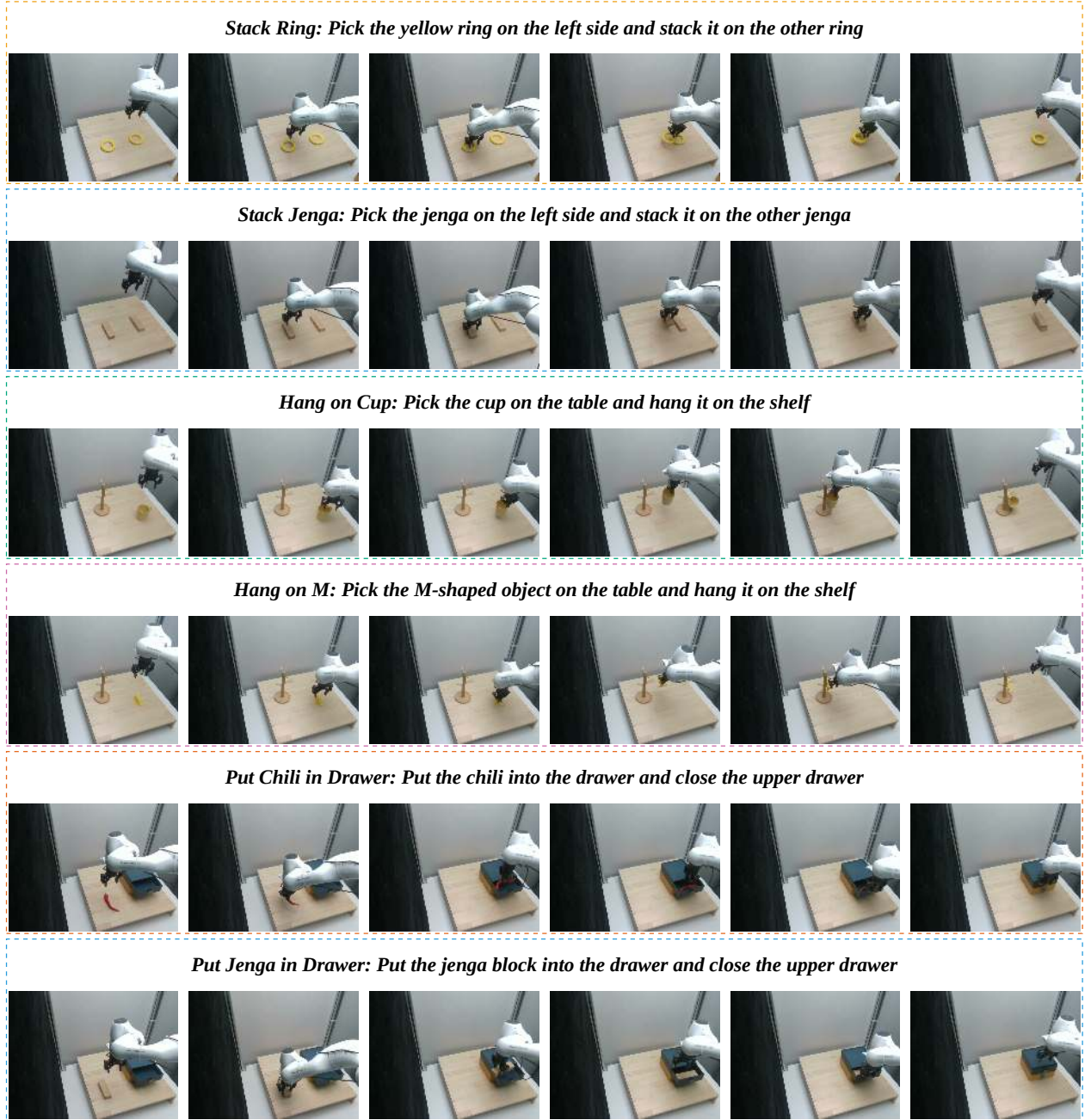


Figure 19 Single-arm task execution sequences on the Franka-Research-3 platform. Each row shows one task, with six frames uniformly sampled from the corresponding left-view rollout video.

Evaluation Protocol. For each single-arm task, we fine-tune the policy with 100 task-specific real-robot demonstrations. After fine-tuning, each task is evaluated over 20 independent real-world trials, and we report the success rate as the number of successful trials out of 20 in [Table 6](#). A stacking trial succeeds only if the left object is picked and stably stacked on the target object. A hanging trial succeeds only if the object is picked and remains hanging on the shelf. A drawer trial succeeds only if the object is placed inside the drawer and the upper drawer is pushed closed.

C.2 Dexterous-Hand Real-Robot Experiments

Task Setup. We evaluate dexterous-hand policies on four real-world manipulation tasks: Stack Toy Tower, Collect Shuttlecocks, Twist off Bottle Cap, and Put Away Clothes. Each task is specified by a natural-language instruction that defines the desired manipulation objective. The task instructions are summarized in Table 12.

Table 12 Task instructions used in the real-world evaluation.

Task	Instruction
Stack Toy Tower	Stack the discs onto the tower pole in order from largest to smallest.
Collect Shuttlecocks	Put all the shuttlecocks into the shuttlecock tube.
Twist off Bottle Cap	Twist off the bottle cap.
Put Away Clothes	Pick up the clothes from the pile on the table and put them into the basket.

All experiments are conducted on a physical dexterous-hand platform. The robot receives the task instruction and executes the manipulation autonomously in the corresponding scene. Figure 20 shows the physical robot, dexterous hand, workspace, camera viewpoint, and representative objects used in the evaluation.



Figure 20 Real-world experimental setup. The figure shows the physical dexterous-hand platform, the workspace, the observation camera, and representative objects for the four manipulation tasks.

For each task, we evaluate the policy under an in-distribution (ID) condition and several out-of-distribution (OOD) conditions. The OOD conditions modify one factor at a time, including object identity, object layout, illumination, or background appearance, while preserving the task instruction and the overall manipulation

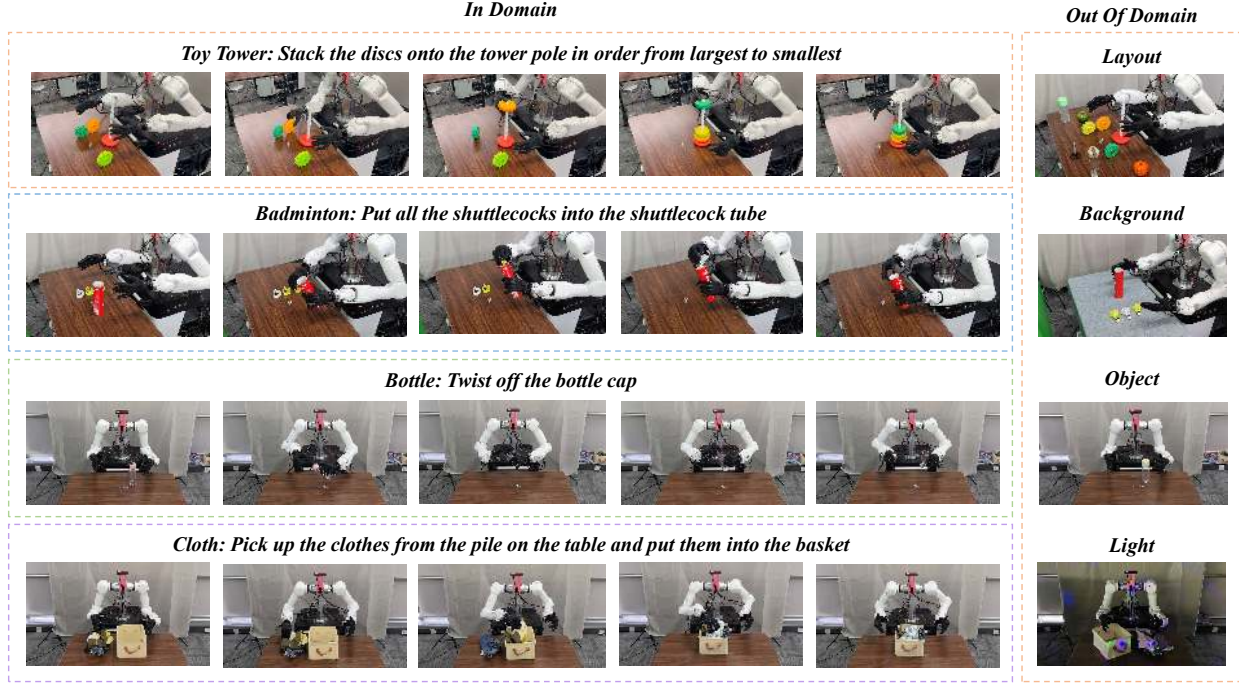


Figure 21 Task execution sequences and evaluation conditions. Each row illustrates the main manipulation stages of one task, while the columns show the corresponding ID scene and OOD variations. OOD conditions include changes in object identity, object layout, illumination, and background appearance.

objective. The task execution sequence and the corresponding ID/OOD configurations are illustrated in Figure 21.

Evaluation Protocol. Each task is evaluated over multiple independent trials under both ID and OOD conditions. We report two complementary metrics: the progress score (Score) and the final success rate (SR).

Final Success Rate. A trial is counted as a final success only when the complete task objective is achieved. The final success rate is computed as

$$S_{\text{final}} = \frac{N_{\text{success}}}{N_{\text{trial}}} \times 100\%,$$

where N_{success} is the number of trials satisfying the complete task criterion and N_{trial} is the total number of valid trials.

Progress Score. The progress score measures the fraction of required manipulation elements that are successfully completed, regardless of whether the final task state is achieved. For trial i , let n_i denote the number of successfully completed elements and m_i denote the total number of elements present in that trial. The progress score is computed as

$$S_{\text{process}} = \frac{\sum_i n_i}{\sum_i m_i} \times 100\%.$$

Task-Specific Criteria.

- **Collect Shuttlecocks.** Each trial contains two to four shuttlecocks on the tabletop. A trial is counted as a final success only when all shuttlecocks are placed into the shuttlecock tube. The progress score is the fraction of shuttlecocks successfully placed into the tube.
- **Stack Toy Tower.** Each trial contains three discs. A trial is counted as a final success only when all three discs are successfully inserted onto the tower pole in descending order of size. The progress score is the fraction of discs successfully inserted.

- **Put Away Clothes.** Each trial contains three to four pieces of clothing. A trial is counted as a final success only when all pieces of clothing are placed into the basket. The progress score is the fraction of clothing items successfully placed into the basket.
- **Twist off Bottle Cap.** A trial is counted as a final success when the bottle cap is fully twisted off and the robot maintains a stable grasp of the bottle or cap. The progress score records whether the cap is successfully twisted off, regardless of whether the final stable grasp is achieved.

C.3 Bimanual Real-Robot Experiments

We conduct a comprehensive evaluation on all 18 tasks of RoboDojo-Real, using the official real-robot evaluation platform provided by the RoboDojo team and covering its three bimanual embodiments: ARX X5, Piper, and Piper X. The evaluation strictly follows the unified protocol defined by the official evaluation team; detailed documentation of the platform and its task specifications is available on the RoboDojo website (<https://robodojo-benchmark.com/>) and in the accompanying official documentation (<https://robodojo-benchmark.com/doc/real-tasks/>).

D Per-Benchmark Simulation Results

The tables below report the full per-benchmark scores summarized in Figure 13, except for LIBERO-Plus, whose scores are already reported in Table 5 of the main text. Within each table the baselines are grouped as VLA or WAM; bold denotes the best value and underline the second best.

Table 13 Evaluation Results on LIBERO. Bold denotes best values, underline second best.

	Spatial	Object	Goal	Long	Avg
VLA					
OpenVLA (Kim et al., 2024)	84.7	88.4	79.2	53.7	76.5
π_0 (Black et al., 2024)	98.0	96.8	94.4	88.4	94.4
StarVLA (Community, 2026)	97.8	98.6	96.2	93.8	96.6
$\pi_{0.5}$ (Physical Intelligence et al., 2025)	98.8	98.2	98.0	92.4	96.9
GR00T-N1.6 (NVIDIA et al., 2025)	97.7	98.5	97.5	94.4	97.0
OpenVLA-OFT (Kim et al., 2025)	97.6	98.4	97.9	94.5	97.1
X-VLA (Zheng et al., 2026b)	98.2	98.6	97.8	97.6	98.1
ABot-M0 (Yang et al., 2026b)	98.8	<u>99.8</u>	99.0	96.6	98.6
Being-H0.5 (Luo et al., 2026a)	99.2	99.6	<u>99.4</u>	97.4	98.9
Qwen-RobotManip (Yuan et al., 2026a)	–	–	–	–	99.2
WAM					
Fast-WAM (Yuan et al., 2026b)	98.2	100.0	97.0	95.2	97.6
Motus (Bi et al., 2026)	96.8	<u>99.8</u>	96.6	97.6	97.7
ImageWAM (Zhang et al., 2026c)	97.2	99.2	98.8	<u>98.4</u>	98.4
LingBot-VA (Li et al., 2026b)	98.5	99.6	97.2	98.5	98.5
DiT4DiT (Ma et al., 2026)	–	–	–	–	98.6
Being-H0.7 (Luo et al., 2026b)	–	–	–	–	99.2
ABot-M0.5 (Chen et al., 2026a)	100.0	<u>99.8</u>	<u>99.4</u>	<u>98.4</u>	99.4
OpenWAM- α	<u>99.6</u>	99.6	99.8	98.2	<u>99.3</u>

Table 14 Evaluation Results on VLABench. Bold denotes best values, underline second best.

	In-dist.			Category			Commonsense			Instruction			Texture			Avg		
	SR	PS	IS	SR	PS	IS	SR	PS	IS	SR	PS	IS	SR	PS	IS	SR	PS	IS
VLA																		
π_0 (Black et al., 2024)	47.0	62.7	67.8	21.2	33.6	44.0	29.1	43.0	54.9	17.3	38.7	58.0	32.2	42.5	50.6	29.4	44.1	55.0
LoHo-Manip (Liu et al., 2026a)	54.0	—	—	23.0	—	—	36.0	—	—	42.0	—	—	39.0	—	—	39.0	—	—
ACoT-VLA (Zhong et al., 2026)	—	66.1	79.8	—	38.9	<u>54.1</u>	—	37.8	52.3	—	39.6	56.8	—	54.6	74.6	—	47.4	63.5
$\pi_{0.5}$ (Physical Intelligence et al., 2025)	65.4	77.8	80.4	38.2	49.7	52.0	43.9	57.3	<u>60.0</u>	48.2	64.2	67.0	44.9	62.3	65.0	48.1	62.3	64.9
ERVLA (Sun et al., 2026)	69.7	81.1	<u>84.2</u>	<u>47.0</u>	<u>61.0</u>	66.4	44.0	55.0	57.2	<u>58.0</u>	<u>70.2</u>	<u>73.8</u>	47.4	62.3	70.6	53.2	65.9	<u>70.4</u>
Xiaomi-Robotics-1 (Team et al., 2026b)	75.6	85.0	79.8	53.0	66.6	66.4	48.4	58.3	58.2	55.8	66.8	70.2	62.6	74.9	<u>74.8</u>	59.1	70.3	69.9
WAM																		
Bridge-WA (Bai et al., 2026)	<u>78.0</u>	<u>85.8</u>	85.0	23.0	28.8	39.0	<u>51.1</u>	<u>64.4</u>	74.2	67.0	80.3	82.0	45.0	60.3	76.0	52.8	64.0	71.2
OpenWAM- α	83.4	87.9	75.2	38.1	45.9	45.9	58.0	64.8	57.9	53.8	64.5	66.5	<u>61.4</u>	<u>72.9</u>	71.6	<u>58.9</u>	<u>67.2</u>	63.5

Table 15 Evaluation Results on RoboTwin2.0-Clean2Random. Bold denotes best values, underline second best.

	Clean	Randomized	Avg
VLA			
StarVLA (Community, 2026)	46.5	3.2	24.9
GR00T-N1.7 (NVIDIA et al., 2025)	43.6	20.7	32.2
X-VLA (Zheng et al., 2026b)	68.0	20.9	44.5
Spatial Forcing (Li et al., 2026a)	77.2	26.7	52.0
ABot-M0 (Yang et al., 2026b)	70.7	36.0	53.4
$\pi_{0.5}$ (Physical Intelligence et al., 2025)	70.7	46.0	58.4
GigaBrain-0.7 (Team et al., 2026a)	66.8	<u>67.9</u>	67.4
Qwen-RobotManip (Yuan et al., 2026a)	<u>84.7</u>	69.4	77.1
WAM			
AHA-WAM (Cai et al., 2026b)	64.3	3.2	33.8
Fast-WAM (Yuan et al., 2026b)	77.8	1.9	39.9
X-WAM (Guo et al., 2026)	70.0	25.8	47.9
4D-WAM (Yang et al., 2026a)	81.5	41.8	61.7
OpenWAM- α	89.4	48.7	<u>69.0</u>

Table 16 Evaluation Results on RoboTwin2.0-Full. Bold denotes best values, underline second best.

	Clean	Randomized	Avg
VLA			
X-VLA (Zheng et al., 2026b)	72.80	72.84	72.82
$\pi_{0.5}$ (Physical Intelligence et al., 2025)	82.70	76.80	79.75
ABot-M0 (Yang et al., 2026b)	86.06	85.08	85.57
Qwen-VLA (Wang et al., 2026a)	86.10	87.20	86.65
StarVLA (Community, 2026)	88.18	88.32	88.25
Galaxea G0.5 (Liu et al., 2026b)	93.70	92.80	93.25
Qwen-RobotManip (Yuan et al., 2026a)	93.70	<u>94.00</u>	<u>93.85</u>
WAM			
Motus (Bi et al., 2026)	88.66	87.02	87.84
Fast-WAM (Yuan et al., 2026b)	91.90	91.80	91.85
LingBot-VA (Li et al., 2026b)	92.93	91.55	92.24
ImageWAM (Zhang et al., 2026c)	93.20	93.56	93.38
LingBot-VA 2.0 (Zhang et al., 2026b)	<u>93.80</u>	93.40	93.60
ABot-M0.5 (Chen et al., 2026a)	94.00	94.20	94.10
OpenWAM-α	93.74	93.46	93.60

Table 17 Evaluation Results on RoboDojo. Bold denotes best values, underline second best.

	Gen-Std		Gen-Rand		Precision		Long-Horizon		Memory		Open		Avg	
	SR	Score	SR	Score	SR	Score	SR	Score	SR	Score	SR	Score	SR	Score
VLA														
StarVLA-α (Ye et al., 2026b)	5.00	7.54	0.00	0.33	4.33	9.90	6.50	14.15	2.44	3.34	0.58	0.68	3.24	6.40
X-VLA (Zheng et al., 2026b)	12.00	17.90	1.00	3.04	12.00	18.32	9.75	16.53	3.56	4.76	0.50	0.55	6.52	10.13
$\pi_{0.5}$ (Physical Intelligence et al., 2025)	15.00	20.93	1.00	5.82	5.50	12.40	14.67	23.54	4.56	5.78	1.67	1.98	6.91	11.41
Spatial Forcing (Li et al., 2026a)	15.00	21.25	4.00	6.98	10.58	17.33	14.58	23.26	4.11	5.43	1.58	1.78	8.04	12.38
Hy-Embodied-0.5-VLA (Zhang et al., 2026a)	17.00	21.98	0.00	1.57	8.00	13.81	14.92	25.74	<u>12.11</u>	<u>13.37</u>	0.58	0.65	8.80	13.07
Xiaomi-Robotics-1 (Team et al., 2026b)	28.00	35.65	6.00	11.44	<u>18.83</u>	<u>26.69</u>	23.67	<u>38.39</u>	6.56	7.81	3.58	3.94	13.93	20.07
Galaxea G0.5 (Liu et al., 2026b)	20.00	26.74	6.00	<u>11.16</u>	20.42	28.25	32.25	44.12	7.33	8.61	1.58	1.73	<u>14.88</u>	<u>20.23</u>
DM0.5 (Dexmal, 2026)	18.00	23.49	4.00	8.06	16.75	24.82	19.50	33.70	47.44	47.74	<u>2.08</u>	<u>2.43</u>	19.34	24.90
WAM														
Fast-WAM (Yuan et al., 2026b)	2.00	4.33	0.00	0.34	0.00	1.96	5.17	9.14	3.44	3.55	0.42	0.42	2.03	3.48
AHA-WAM (Cai et al., 2026b)	6.00	10.32	0.00	1.26	2.42	5.86	2.67	8.61	2.78	2.97	0.83	0.88	2.39	4.82
GigaWorld-Policy (Ye et al., 2026a)	6.00	10.28	0.00	0.41	1.83	6.15	8.92	15.51	2.22	3.46	0.50	0.54	3.27	6.20
X-WAM (Guo et al., 2026)	5.00	11.24	1.00	3.54	1.83	6.72	9.08	17.47	4.67	6.32	0.25	0.57	3.83	7.69
OpenWAM-α	<u>25.56</u>	<u>33.16</u>	<u>4.11</u>	8.26	9.25	18.45	<u>25.33</u>	34.93	9.11	10.41	1.08	1.41	11.92	17.18

Table 18 Evaluation Results on EBench. Bold denotes best values, underline second best.

	Table Top		Simple PnP		Long Horizon		Overall	
	SR	Score	SR	Score	SR	Score	SR	Score
VLA								
StarVLA-OFT (Community, 2026)	–	–	–	–	–	–	0.0	0.2
π_0 (Black et al., 2024)	15.7	30.0	35.0	39.0	17.0	41.0	23.6	37.0
X-VLA (Zheng et al., 2026b)	8.6	24.0	50.0	54.0	6.2	25.0	23.7	36.0
InternVLA-A1 (Cai et al., 2026a)	4.3	11.0	43.0	47.0	17.9	46.0	23.9	36.0
$\pi_{0.5}$ (Physical Intelligence et al., 2025)	12.9	32.0	45.0	50.0	18.1	39.0	27.1	41.0
GigaBrain-0.7 (Team et al., 2026a)	–	–	–	–	–	–	33.3	46.0
Qwen-RobotManip (Yuan et al., 2026a)	50.0	70.0	<u>56.5</u>	<u>60.0</u>	<u>29.9</u>	<u>55.0</u>	<u>45.6</u>	<u>60.0</u>
WAM								
Fast-WAM (Yuan et al., 2026b)	–	–	–	–	–	–	4.7	7.6
OpenWAM- α	<u>30.0</u>	<u>44.2</u>	67.5	72.0	44.3	72.6	49.4	64.7

Table 19 Evaluation Results on RoboCasa365. Bold denotes best values, underline second best.

	Atomic	Comp.-Seen	Comp.-Unseen	Avg
VLA				
Diffusion Policy (Chi et al., 2025)	15.7	0.2	1.3	6.1
π_0 (Black et al., 2024)	36.3	5.2	0.7	15.0
$\pi_{0.5}$ (Physical Intelligence et al., 2025)	39.6	7.1	1.2	16.9
GR00T-N1.5 (NVIDIA et al., 2025)	50.7	14.8	2.7	23.9
Qwen-RobotManip (Yuan et al., 2026a)	68.6	20.1	<u>14.9</u>	35.9
RLDX-1 (Kim et al., 2026a)	67.6	27.9	8.5	36.0
Xiaomi-Robotics-1 (Team et al., 2026b)	80.2	57.1	32.1	57.4
WAM				
GigaWorld-Policy (Ye et al., 2026a)	44.4	11.8	2.9	20.7
ABot-M0.5 (Chen et al., 2026a)	<u>75.9</u>	<u>38.3</u>	2.7	<u>40.4</u>
OpenWAM- α	69.7	32.1	8.9	38.2

Table 20 Evaluation Results on RoboCasa-GR1. Bold denotes best values, underline second best.

	SR (%)
VLA	
π_0 (Black et al., 2024)	13.6
$\pi_{0.5}$ (Physical Intelligence et al., 2025)	37.0
GR00T-N1.5 (NVIDIA et al., 2025)	48.0
StarVLA (Community, 2026)	48.8
GR00T-N1.6 (NVIDIA et al., 2025)	49.9
VP-VLA (Wang et al., 2026c)	53.8
Being-H0.5 (Luo et al., 2026a)	53.9
Qwen-VLA-Instruct (Wang et al., 2026a)	56.7
RLDX-1 (Kim et al., 2026a)	58.7
PhysBrain 1.0 (Lin et al., 2026b)	64.5
WAM	
UWM (Zhu et al., 2025)	20.0
Being-H0.7 (Luo et al., 2026b)	49.2
DiT4DiT (Ma et al., 2026)	50.8
LDA-1B (Lyu et al., 2026)	55.4
OpenWAM- α	<u>60.5</u>